INDUSTRIAL PROJECT

on

VStack – AI Powered Website Generator with Client-Specific Containerized Environment

Submitted in partial fulfillment of requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS
in
DEPARTMENT OF COMPUTER APPLICATION
Engineering College Bikaner

Session-2024-2026

Submitted by:
VAIBHAV KHATRI

(Signature)

Name: Vaibhav khatri

Father's Name: Mahesh Kumar

Roll No.:24CEBMC625
Semester: IV

Guided by:
DR. PAWAN TANWAR

(Signature)

Name: Dr. Pawan TANWAR

MCA Department
Engineering College Bikaner
Bikaner Technical University, Bikaner, Rajasthan.

Submitted to:
DR.SANJAY TEJASVEE

(Signature)

Name: Dr. Sanjay Tesjasvee

HOD, MCA Department
Engineering College Bikaner
Bikaner Technical University, Bikaner, Rajasthan.

Engineering College Bikaner,

Pugal Road, RIICO Karni Industrial Area, Bikaner, Rajasthan-33004
web:www.ecb.ac.in, Telephone: +91-1512253404,

Bikaner Technical University,
UCET Campus
Pugal Road, RIICO Karni Industrial Area, Bikaner, Rajasthan-33004
web:www.btu.ac.in

CERTIFICATE FROM GUIDE

This is to certify that the Industrial Project entitled "AI-Powered Web Development Platform with Cloud IDE (VStack & Cloud IDE)" has been submitted by the undersigned student in partial fulfillment of the requirements for the award of Master of Computer Applications degree from Bikaner Technical University, Bikaner, Rajasthan.

This project work has been carried out under my supervision and guidance, and the work is original and has not been submitted elsewhere for the award of any other degree or diploma.

The project demonstrates a high level of technical competence and innovative approach in developing an AI-powered website generator integrated with a full-featured cloud-based IDE using Docker containerization and reverse proxy technology.

Date: 27/02/26

Internal Guide / Mentor

Name: Dr . Pawan Tanwar

Designation: MCA DEPARTMENT

MCA Department

Engineering College Bikaner

Bikaner Technical University, Bikaner, Rajasthan

CERTIFICATE OF ORIGINALITY

We hereby declare that the work presented in this Industrial Project Report entitled "AI-Powered Web Development Platform with Cloud IDE (VStack & Cloud IDE)" submitted to the Department of Computer Application, Engineering College Bikaner, Bikaner Technical University, Rajasthan, for the award of the degree of Master of Computer Applications is an authentic record of our own work carried out during the period of the Industrial Project.

The matter embodied in this report has not been submitted by us for the award of any other degree or diploma to this or any other university or institute.

All sources of information have been duly acknowledged and the work contained in this report is original to the best of our knowledge and belief.

Date: 27/02/26 Place: Engineering College Bikaner

Student Signature: Vaibhav

Name: VAIBHAV KHATRI

Roll No.: 24CEBMC625

Semester: IV

Countersigned by:

HOD, MCA Department,Engineering College Bikaner

CDTS
Cloud & DevOps

Certificate of Completion

This is to certify that Mr.Vaibhav Khatri S/o Sh. Mahesh Kumar, student of MCA from Engineering College Bikaner, Bikaner has successfully completed an industrial training cum internship program on **"DevOps & AWS Cloud"** and also worked on the same project.

The duration of internship was 45 days from 19.01.2026 to 05.03.2026

During the internship session his behaviour was good.

CDTS BIKANER

NARENDRA SINGH BHUI
Academy Director

Certificate number: CDTS/CERT/Gen-File/258
Issued date: 05.03.2026

URL: https://www.cdtsbikaner.in Contact: +91-7742163889
Behind Rajasthan Patrika Press, Amar Singh Pura, Bikaner PIN 334001 (Rajasthan) INDIA
Society Reg. No. COOP/2020/BIKANER/200308

Society Reg No: COOP/2020/BIKANER/200308

CDTS Cloud & DevOps

Centre for Cloud & DevOps Training Society, Bikaner

(A Complete IT Solutions for Open-Source Training and Software Development)
Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner-334001. Contact: +91-7742163889
Website: https://www.cdtsbikaner.in , Email: info@cdtsbikaner.com , cdtsbikaner@gmail.com

F.No. /CDTS/ 2026/MCA -Internship/158 Date: 05-03-2026

TO WHOM IT MAY CONCERN

This is to certify that Mr. Vaibhav Khatri S/o Sh. Mahesh Kumar, student of MCA from Engineering College Bikaner, Bikaner (Rajasthan) has successfully completed an industrial training cum internship program on **"DevOps & AWS Cloud"** and also worked on the same projects. The duration of the internship was 45 days between 19.01.2026 to 05-03-2026.

During the period of his training program with us found punctual, hardworking, and inquisitive.

We wish him success in life.

Narendra Singh Bhui
(Director)

CDTS BIKANER

ACKNOWLEDGEMENT

It is with great pleasure and immense gratitude that I present this Industrial Project Report. The successful completion of this project would not have been possible without the invaluable support, guidance, and encouragement of numerous individuals.

First and foremost, I would like to express my sincere and profound gratitude to my Internal Guide and Mentor at the Department of Computer Application, Engineering College Bikaner, for their unwavering support, insightful suggestions, and patient guidance throughout the course of this project. Their expertise in full-stack development and cloud technologies has been an invaluable resource.

I am deeply thankful to the Head of the Department of Computer Application, Engineering College Bikaner, for providing the necessary infrastructure, laboratory facilities, and an inspiring academic environment that facilitated the execution of this project.

My heartfelt thanks go to all the faculty members of the MCA Department who shared their knowledge and provided guidance whenever required. Their teachings have formed the technical foundation upon which this project is built.

I would also like to acknowledge the open-source community whose libraries, frameworks, and tools — including Next.js, Tailwind CSS, Drizzle ORM, Better Auth, Google Gemini AI, Docker, Traefik, xterm.js, Monaco Editor, and Supabase — have been instrumental in the development of both VStack and the Cloud IDE platform.

Last but not least, I owe a debt of gratitude to my family and friends whose constant encouragement, moral support, and belief in my abilities kept me motivated throughout this challenging endeavor.

Student Name: VAIBHAV KHATRI

Department of Computer Application

Engineering College Bikaner

ABSTRACT

This Industrial Project presents the design, development, and deployment of two interrelated and innovative web-based software platforms: VStack, an AI-powered website generator, and Cloud IDE, a browser-accessible cloud-based Integrated Development Environment. Together, these platforms form a comprehensive ecosystem for modern web development, enabling users to generate, preview, edit, and deploy web applications entirely from a web browser, without requiring any local development environment setup.

VStack harnesses the power of Google Gemini AI to generate complete, functional React-based website code from simple natural language prompts. The platform integrates with Supabase (PostgreSQL) for persistent data storage, Drizzle ORM for type-safe database interactions, Better Auth for secure authentication, and the PayPal payment gateway for token-based monetization. Users can describe the website they envision in plain English, and VStack produces production-ready code within seconds, which is immediately rendered in an embedded sandbox environment powered by CodeSandbox's Sandpack.

Cloud IDE complements VStack by providing a fully-featured web-based coding environment. Built as a monorepo using TurboRepo and managed with pnpm, it consists of a Next.js frontend, a Node.js backend WebSocket server, a Traefik reverse proxy, and per-user Docker containers. Each user who connects to the Cloud IDE receives a dedicated, isolated Docker container running a bash environment. The xterm.js library emulates a full terminal interface in the browser, while the Monaco Editor (the engine behind VS Code) provides syntax-highlighted code editing. The Traefik reverse proxy dynamically routes requests to the correct

user container based on the subdomain (e.g., userid.localhost), enabling true multi-user isolation.

The combined platform addresses critical industry challenges such as environment inconsistency (the 'works on my machine' problem), onboarding friction for new developers, and the growing demand for AI-assisted development tools. The architecture is scalable, secure, and aligned with modern DevOps and cloud-native principles.

TABLE OF CONTENTS

Section	Title	Page No.
	Title Page	1
	Certificate from Guide	2
	Certificate of Originality	3
	Acknowledgement	4
	Abstract	5
	Table of Contents	6
	List of Tables	7
	List of Figures	7
Chapter 1	Introduction	8
1.1	Company / Educational Institute Profile	8
1.2	Existing System and Need for System	12
1.3	Scope of Work	16
1.4	Operating Environment – Hardware and Software	19
Chapter	Proposed System	22

Section	Title	Page No.
2		
2.1	Proposed System Overview	22
2.2	Objectives of System	27
2.3	User Requirements	31
Chapter 3	Analysis and Design	36
3.1	Entity Relationship Diagram (ERD)	36
3.2	System Architecture	40
3.3	Database Requirements and User Interfaces	46
3.4	Data Flow Diagram (DFD)	52
3.5	Data Dictionary	57
3.6	Table Design	61
3.7	Code Design	65
3.8	Menu Screens	70
3.9	Input Screens	73
3.10	Report Formats	75
3.11	Test Procedures and Implementation	77
Chapter 4	User Manual	83

Section	Title	Page No.
4.1	User Manual	83
4.2	Operations Manual / Menu Explanation	87
4.3	Forms and Report Specifications	90
	Drawbacks and Limitations	93
	Proposed Enhancements	95
	Conclusions	97
	Bibliography	99
Annexure 1	Input Forms with Data	101
Annexure 2	Output Reports with Data	103
Annexure 3	Sample Code Snippets	105

LIST OF TABLES

Table No.	Title	Page
Table 1.1	Hardware Requirements	19
Table 1.2	Software Requirements	20
Table 1.3	Comparison of Existing Systems	14
Table 2.1	Token System Configuration	25
Table 2.2	Functional Requirements	32
Table 2.3	Non-Functional Requirements	34
Table 3.1	Users Table Schema	61
Table 3.2	Chats Table Schema	62
Table 3.3	Sessions Table Schema	63
Table 3.4	Accounts Table Schema	64
Table 3.5	Test Cases – Unit Testing	78
Table 3.6	Test Cases – Integration Testing	80
Table 3.7	Test Cases – System Testing	81

LIST OF FIGURES

Figure No.	Title	Page
Figure 1.1	VStack Platform Overview	9
Figure 1.2	Cloud IDE Architecture Overview	11
Figure 2.1	AI Website Generation Workflow	23
Figure 2.2	Docker Container Lifecycle	26
Figure 3.1	Entity Relationship Diagram – VStack	37
Figure 3.2	Entity Relationship Diagram – Cloud IDE	39
Figure 3.3	System Architecture – VStack	41
Figure 3.4	System Architecture – Cloud IDE	44
Figure 3.5	DFD Level 0 – VStack	53
Figure 3.6	DFD Level 1 – AI Code Generation	54
Figure 3.7	DFD Level 0 – Cloud IDE	55
Figure 3.8	DFD Level 1 – Container Management	56
Figure 3.9	Menu Structure – VStack	71
Figure 3.10	Menu Structure – Cloud IDE	72

CHAPTER 1: INTRODUCTION

The rapid evolution of web development technologies has fundamentally transformed how software is built, deployed, and maintained. Modern developers require increasingly sophisticated tools that not only support the full software development lifecycle but also reduce friction in environment setup, collaboration, and deployment. The emergence of cloud computing, containerization, and artificial intelligence has opened unprecedented opportunities for innovation in developer tooling.

This Industrial Project presents two interconnected systems — VStack and Cloud IDE — that together form a comprehensive, browser-based, AI-augmented web development ecosystem. Developed as a final-year MCA Industrial Project at Engineering College Bikaner under Bikaner Technical University, this project demonstrates the practical application of advanced computer science concepts in solving real-world software development challenges.

1.1 Company / Educational Institute Profile

Engineering College Bikaner

Engineering College Bikaner (ECB) is a premier engineering and technology institution located at Pugal Road, RIICO Karni Industrial Area, Bikaner, Rajasthan — 334004. Affiliated with Bikaner Technical University (BTU), ECB is committed to providing high-quality technical education that bridges the gap between academic learning and industry requirements.

The Department of Computer Application at ECB offers a comprehensive Master of Computer Applications (MCA) programme that covers a wide spectrum of computing disciplines including software engineering, database management, web technologies, cloud computing, artificial intelligence, and systems programming. The MCA programme at ECB is structured to equip students with both theoretical knowledge and hands-on practical skills demanded by the modern IT industry.

Attribute	Details
Institution Name	Engineering College Bikaner (ECB)
Address	Pugal Road, RIICO Karni Industrial Area, Bikaner, Rajasthan - 334004
Website	www.ecb.ac.in
Telephone	+91-0000000
Affiliated University	Bikaner Technical University (BTU), Bikaner
Department	Department of Computer Application
Programme	Master of Computer Applications (MCA)
Programme Duration	2 Years (4 Semesters)
Vision	To be a centre of excellence for technical education and research
Mission	To nurture technically competent, ethically responsible professionals

About the Project

This Industrial Project was conceptualized, designed, and developed at the Incubation Centre and Computer Laboratory of the Department of Computer Application, Engineering College Bikaner. The project was undertaken as an extension of the student's academic learning and is of strong industrial significance, addressing real-world demands for AI-powered development tools and cloud-based IDEs — both of which are rapidly growing sectors of the global software industry.

The two platforms developed — VStack and Cloud IDE — represent a complete, end-to-end web development ecosystem. VStack focuses on the front-end of the development lifecycle (AI-driven code generation and preview), while Cloud IDE addresses the full development loop by providing an isolated, containerized coding environment directly in the browser.

VStack — AI-Powered Website Generator

VStack is a modern web application that leverages Google Gemini AI to generate complete, functional React-based websites from natural language descriptions. Built with Next.js 16, Tailwind CSS, Supabase, Drizzle ORM, Better Auth, and PayPal integration, it provides a seamless experience from idea to code in seconds.

Cloud IDE — Web-Based Cloud Integrated Development Environment

Cloud IDE is a browser-accessible cloud development environment built as a TurboRepo monorepo. It uses Next.js for the frontend, Node.js with Socket.IO for real-time communication, node-pty for terminal process management, Docker (via Dockerode) for containerized user environments, Traefik as a dynamic reverse proxy, Monaco Editor for file editing, and xterm.js for terminal emulation.

1.2 Existing System and Need for System

Overview of Existing Systems

The web development tooling landscape currently offers a variety of solutions, but each comes with significant limitations that this project aims to address.

Existing AI Website Generators

Several platforms offer AI-assisted web development, including Wix ADI, Webflow AI features, and experimental tools from Vercel and Netlify. However, the majority of these are proprietary, locked into specific hosting ecosystems, non-programmable, and do not expose the generated code for further development. They are primarily aimed at non-technical users and lack integration with proper development environments.

Existing Cloud IDEs

Cloud IDEs such as GitHub Codespaces, Replit, CodeSandbox, and StackBlitz have gained significant traction. However, these platforms are predominantly commercial with restrictive free tiers, limited customizability at the infrastructure level, and do not allow self-hosting or open-source deployment. They do not integrate AI website generation as a core feature.

Feature	VStack	Wix ADI	Webflow AI	GitHub Codespaces	Replit	Cloud IDE (This Project)
AI Code Generation	Yes (Gemini AI)	Yes (Proprietary)	Limited	No	No	No (IDE only)

Feature	VStack	Wix ADI	Webflow AI	GitHub Codespaces	Replit	Cloud IDE (This Project)
Code Access & Edit	Yes (Sandpack)	No	Limited	Yes	Yes	Yes (Monaco)
Terminal Access	No	No	No	Yes	Yes	Yes (xterm.js)
Docker Containers	No	No	No	Yes	No	Yes (per-user)
Self-Hostable	Yes (Next.js)	No	No	No	No	Yes
Open Source	Yes	No	No	No	Partial	Yes
Token/Payment System	Yes (PayPal)	Subscription	Subscription	Usage-based	Subscription	N/A
Real-time Preview	Yes (Sandpack)	Yes	Yes	Partial	Yes	Via Proxy

---

Need for the System

The following critical gaps in existing solutions motivate the development of this project:

- No single platform currently combines AI website generation with a full cloud IDE experience.
- Existing AI generators do not produce code that can be directly edited in a professional IDE environment.
- Cloud IDEs are expensive for individual developers and students; self-hosted alternatives are rare.
- Environment inconsistency ('works on my machine' problem) remains a critical issue in collaborative development.
- There is no open, extensible platform that allows educators and institutions to deploy their own cloud IDE.
- Current tools lack seamless integration between AI code generation and container-based code execution.

This project fills these gaps by delivering a unified, open-source, self-hostable platform that combines Gemini AI-powered code generation (VStack) with Docker-based, browser-accessible development environments (Cloud IDE).

1.3 Scope of Work

The scope of this Industrial Project encompasses the end-to-end design, development, testing, and documentation of two integrated software platforms. The project scope is defined across the following dimensions:

VStack — Scope

VStack covers the following functional areas:

- User registration, authentication, and session management using Better Auth with email/password and social login.

- AI-powered website code generation using Google Gemini AI, producing React component code from natural language descriptions.
- Real-time code preview using CodeSandbox Sandpack embedded in the application.
- Persistent storage of chat sessions, generated code, and file structures in Supabase (PostgreSQL) via Drizzle ORM.
- Token-based usage management: each AI generation consumes tokens; tokens can be purchased via PayPal.
- A suggestion-based home page for quick code generation prompts.
- Responsive, modern UI built with Tailwind CSS, Radix UI primitives, Framer Motion animations, and Lucide icons.
- Multi-file project structure support, with generated files organized in a file-tree pattern.

Cloud IDE — Scope

Cloud IDE covers the following functional areas:

- A Next.js web frontend providing the IDE user interface.
- Docker container management: starting, stopping, and monitoring per-user containers using Dockerode.
- A WebSocket server (Socket.IO) running inside each Docker container, providing real-time file events and terminal I/O.
- Terminal emulation using node-pty (pseudo-terminal) connected to a bash shell inside the container.
- Monaco Editor integration for browser-based file editing with syntax highlighting.
- File explorer component showing the container's file system tree, supporting file browsing and editing.
- Traefik reverse proxy for dynamic, label-based routing of requests to the correct per-user container.
- TurboRepo monorepo structure managing the client, server, and traefik packages.

Out of Scope

The following items are outside the scope of this project but may be considered for future enhancement:

- Mobile application versions of either platform.
- Multi-user collaborative editing (real-time pair programming).
- Git version control integration within Cloud IDE.
- Cloud deployment automation (CI/CD pipelines) for generated VStack code.
- Support for programming languages other than JavaScript/TypeScript/React in the AI generation model.

1.4 Operating Environment – Hardware and Software

Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 / AMD Ryzen 3 (2 GHz)	Intel Core i5/i7 / AMD Ryzen 5/7 (3+ GHz)
RAM	4 GB	8 GB – 16 GB
Storage	20 GB free SSD	50 GB+ SSD
Network	10 Mbps stable internet	100 Mbps broadband
Display	1024 x 768 resolution	1920 x 1080 Full HD
Server (Cloud IDE)	2 vCPU, 4 GB RAM VM	4 vCPU, 8 GB RAM VM with Docker support
Docker Host	Linux host with Docker	Linux with Docker 24+, 50 GB disk

Component	Minimum Requirement	Recommended Requirement
	20+	

Software Requirements

Software / Technology	Version	Purpose	Platform
Node.js	v20.x LTS	JavaScript runtime	All
pnpm	v10.x	Package manager	All
Next.js	16.1.6	React framework for both apps	All

Software / Technology	Version	Purpose	Platform
TypeScript	5.x	Type-safe development	All
Tailwind CSS	3.x	Utility-first CSS framework	All
Google Gemini AI	API v1	AI website generation (VStack)	VStack
Better Auth	1.5.x	Authentication system (VStack)	VStack
Drizzle ORM	0.45.x	Database ORM (VStack)	VStack
Supabase	Latest	PostgreSQL hosting (VStack)	VStack
PayPal SDK	Latest	Payment gateway (VStack)	VStack
CodeSandbox Sandpack	2.19.x	In-browser code execution (VStack)	VStack
Docker Engine	24.x	Container management	Cloud IDE
Dockerode	Latest	Docker Node.js API client	Cloud IDE
Traefik	v2.9	Reverse proxy	Cloud IDE
Socket.IO	4.x	WebSocket communication	Cloud IDE

Software / Technology	Version	Purpose	Platform
node-pty	Latest	Pseudo-terminal process	Cloud IDE
xterm.js	5.x	Browser terminal emulator	Cloud IDE
Monaco Editor	Latest	In-browser code editor	Cloud IDE
TurboRepo	Latest	Monorepo build system	Cloud IDE
PostgreSQL	15.x	Relational database	VStack
Web Browser	Chrome 120+ / Firefox 120+	Client access	All
Linux OS	Ubuntu 22.04 LTS	Server/Docker host	Cloud IDE
Git	2.x	Version control	All

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System

The proposed system is a dual-platform, browser-centric, AI-augmented web development ecosystem. It addresses the complete software development lifecycle for web applications: from idea generation through AI-powered code creation, to hands-on coding in a cloud-hosted isolated environment.

VStack — Proposed System

VStack is proposed as a cloud-native, AI-first website generator accessible entirely from a web browser. The system accepts natural language input from authenticated users, processes it through Google Gemini AI with carefully engineered prompts, and returns a complete, functional React-based website — both as a visual preview and as editable source code.

The architecture is built on Next.js 16 with the App Router paradigm, enabling server-side rendering, server actions, and API routes within a single unified codebase. Authentication is managed by Better Auth with support for email/password and OAuth providers. The database layer uses Supabase-hosted PostgreSQL accessed through Drizzle ORM with strict TypeScript types enforced at the schema level.

The AI integration uses two distinct Gemini sessions: a chat session for conversation-style interaction that guides the user in refining their requirements, and a code session that produces structured JSON output containing the complete file structure of the generated website. This dual-session approach ensures that the

conversational UX remains fluid while the code generation remains reliable and parseable.

VStack System Flow

1. User visits VStack homepage and sees suggestion prompts.
2. User types a website description or selects a suggestion.
3. System checks authentication — unauthenticated users are redirected to sign-in.
4. A new chat session is created with a unique chat ID.
5. The user message is sent to the Gemini chat session with a React chat prompt.
6. Gemini returns a conversational response explaining the generated website.
7. Simultaneously, the user message is sent to the Gemini code session.
8. Gemini returns a JSON object containing the complete file structure.
9. Files are parsed and stored in the database via Drizzle ORM.
10. The Sandpack sandbox renders the generated React code as a live preview.
11. User can view, edit code files, and iterate with new prompts.
12. Token count is decremented per generation; user can purchase more via PayPal.

Cloud IDE — Proposed System

Cloud IDE is proposed as a self-hosted, multi-user, Docker-based cloud development environment. Each user who accesses the IDE receives a dedicated, isolated Docker container running a Node.js-based WebSocket server with node-pty for terminal access. The Next.js frontend communicates with the container via a dynamic reverse proxy (Traefik) that routes requests based on the user ID embedded in the subdomain.

Cloud IDE System Flow

13. User opens the Cloud IDE web application.

14. Frontend checks if a container for this user is already running via the management API.
15. If no container exists, the frontend calls /start to create and launch a new Docker container.
16. The container starts with the cloud-ide image and is labeled with Traefik routing rules.
17. Traefik picks up the labels and registers a new route: userid.localhost -> container port 9000.
18. The WebSocket server inside the container starts and begins listening.
19. The frontend connects to the container's WebSocket endpoint via the Traefik proxy.
20. File tree is fetched from the container's /files endpoint and displayed in the file explorer.
21. User selects files to view/edit in Monaco Editor; changes are saved back to the container.
22. User interacts with the terminal; commands are sent to node-pty and output is returned via Socket.IO.
23. File watcher (chokidar) detects file changes and emits events to update the frontend.
24. User can stop the container session when done.

Feature	VStack	Cloud IDE
Primary Purpose	AI website generation & preview	Cloud development environment
AI Integration	Google Gemini AI	None (IDE tool)
Code Editor	Sandpack (CodeSandbox)	Monaco Editor (VS Code engine)
Terminal	None	xterm.js + node-pty + bash

Feature	VStack	Cloud IDE
Persistence	Supabase PostgreSQL	Docker volume / container filesystem
Multi-user Isolation	Session-based (auth)	Per-container Docker isolation
Real-time Updates	Chat-based AI iteration	Socket.IO file events + terminal I/O
Reverse Proxy	Next.js built-in routing	Traefik dynamic routing
Authentication	Better Auth (email/OAuth)	Not yet implemented (planned)
Payments	PayPal token system	None

2.2 Objectives of System

The objectives of the proposed system are defined at three levels: primary objectives (core deliverables), secondary objectives (supporting features), and quality objectives (non-functional goals).

Primary Objectives — VStack

- Provide a conversational AI interface that accepts natural language website descriptions and produces complete, functional React code using Google Gemini AI.
- Enable instant, in-browser preview of generated websites using CodeSandbox Sandpack with zero setup required from the user.
- Implement a secure, scalable user authentication system using Better Auth supporting email/password and social login.

- Establish a robust, type-safe data persistence layer using Drizzle ORM with Supabase-hosted PostgreSQL for storing user sessions, chat history, and generated file structures.
- Integrate a PayPal-based payment gateway to support a token-based monetization model, enabling sustainable operation of the platform.
- Deliver a highly responsive, accessible, and visually appealing user interface using Tailwind CSS, Radix UI, and Framer Motion.

Primary Objectives — Cloud IDE

- Provide a fully functional, browser-accessible Integrated Development Environment with file exploration, code editing, and terminal access.
- Implement per-user Docker container isolation ensuring that each user's development environment is independent, secure, and reproducible.
- Enable dynamic routing of user requests to their specific containers using Traefik reverse proxy with label-based auto-configuration.
- Provide a professional-grade code editing experience using Monaco Editor (the engine that powers Visual Studio Code).
- Deliver a real-time terminal experience using xterm.js connected to a node-pty pseudo-terminal running bash inside the container.
- Implement real-time file system event propagation using chokidar and Socket.IO for live file tree updates.

Secondary Objectives

- Maintain a clean, modular, and well-documented codebase following TypeScript best practices.
- Ensure both platforms are self-hostable and open-source, promoting transparency and community contributions.
- Design both systems with scalability in mind, enabling horizontal scaling of container hosts and AI processing.

- Implement comprehensive error handling and user-friendly error messages throughout both platforms.
- Support multiple file types in the Cloud IDE file explorer with appropriate syntax highlighting.

Quality Objectives

- Response time for AI code generation: under 10 seconds for typical prompts.
- Container startup time: under 5 seconds from request to ready state.
- Terminal latency: under 100 milliseconds for command echo.
- System uptime: 99.5% target availability.
- Security: no cross-user container access; all authentication tokens properly managed.

2.3 User Requirements

User requirements have been gathered through analysis of target user groups, study of competing platforms, and iterative design thinking. They are categorized as Functional Requirements (what the system does) and Non-Functional Requirements (how well it does it).

User Personas

Three primary user personas have been identified:

- Student Developer: An MCA/BCA student learning web development who needs a low-friction way to experiment with React and modern tooling without complex setup.
- Freelance Developer: A professional who needs to quickly prototype websites for clients using AI assistance, then refine the code in a proper editor.
- Educator / Instructor: A faculty member who wants to provide students with consistent, reproducible development environments without managing individual machine configurations.

Functional Requirements — VStack

Req. ID	Requirement	Priority
FR-V-01	User shall be able to register with email and password or via social (Google/GitHub) OAuth.	High
FR-V-02	User shall be able to log in and maintain an authenticated session across page refreshes.	High
FR-V-03	User shall be able to type a natural language website description and submit it for AI generation.	High
FR-V-04	System shall call Gemini AI chat session and return a conversational response.	High
FR-V-05	System shall call Gemini AI code session and return a structured JSON file tree.	High
FR-V-06	System shall render the generated React code in a live Sandpack preview.	High
FR-V-07	User shall be able to view and edit individual generated files in a code editor.	High
FR-V-08	System shall persist chat history and generated files to the Supabase database.	High
FR-V-09	System shall enforce a token limit per user, with each generation consuming one token.	Medium
FR-V-10	User shall be able to purchase additional tokens via PayPal payment gateway.	Medium
FR-V-	System shall display quick-start suggestion	Low

Req. ID	Requirement	Priority
11	prompts on the homepage.	
FR-V-12	System shall display a sidebar with the user's previous chat sessions.	Medium
FR-V-13	User shall be able to navigate to and reload previous chat sessions.	Medium

Functional Requirements — Cloud IDE

Req. ID	Requirement	Priority
FR-C-01	System shall start a per-user Docker container on user connection if none exists.	High
FR-C-02	System shall stop and remove a container when the user terminates the session.	High
FR-C-03	System shall route user requests to the correct container via Traefik reverse proxy.	High
FR-C-04	Frontend shall display a file tree explorer showing the container's filesystem.	High

Req. ID	Requirement	Priority
FR-C-05	User shall be able to open files and view their content in Monaco Editor.	High
FR-C-06	User shall be able to edit files in Monaco Editor and save changes to the container.	High
FR-C-07	Frontend shall display an embedded terminal emulated by xterm.js.	High
FR-C-08	User shall be able to type commands in the terminal which execute in the container bash.	High
FR-C-09	File changes in the container shall be reflected in real-time in the file explorer.	Medium
FR-C-10	System shall provide a health check endpoint for container status monitoring.	Medium
FR-C-11	Management API shall expose /start, /running, and /stop endpoints.	High
FR-C-12	System shall support multiple simultaneous users each with isolated containers.	High

Non-Functional Requirements

Req. ID	Category	Requirement	Metric
NFR-01	Performance	AI generation response time	< 10 seconds (typical)
NFR	Performa	Container startup time	< 5 seconds

Req. ID	Category	Requirement	Metric
-02	nce		
NFR-03	Performance	Terminal input-to-display latency	< 100 ms
NFR-04	Performance	File tree load time	< 2 seconds
NFR-05	Scalability	Simultaneous users	≥ 50 concurrent users
NFR-06	Security	Cross-user container access	Zero tolerance
NFR-07	Security	API authentication	All AI endpoints require auth
NFR-08	Usability	Browser compatibility	Chrome 120+, Firefox 120+
NFR-09	Reliability	System uptime	≥ 99.5%
NFR-10	Maintainability	Code coverage	TypeScript strict mode throughout
NFR-11	Portability	Deployment	Docker-compose single-command deploy
NFR-12	Accessibility	UI responsiveness	Fully responsive (mobile–desktop)

CHAPTER 3: ANALYSIS AND DESIGN

3.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram describes the data entities involved in the system and the relationships between them. Two separate ERDs are presented: one for VStack (which has a database layer) and one for Cloud IDE (which manages container lifecycle without a traditional relational database).

VStack ERD — Entities and Attributes

Entity: USER

Attribute	Type	Constraint	Description
id	TEXT	PRIMARY KEY	Unique user identifier (UUID)
name	TEXT	NOT NULL	User's display name
email	TEXT	NOT NULL, UNIQUE	User's email address
emailVerified	BOOLEAN	NOT NULL, DEFAULT false	Email verification status
image	TEXT	NULLABLE	Profile image URL
createdAt	TIMESTAMP	NOT NULL	Account creation timestamp
updatedAt	TIMEST	NOT NULL	Last profile update timestamp

Attribute	Type	Constraint	Description
	AMP		
tokens	INTEGER	DEFAULT 10	AI generation token balance

Entity: CHAT

Attribute	Type	Constraint	Description
id	SERIAL	PRIMARY KEY	Auto-incremented chat identifier
chatId	TEXT	NOT NULL, UNIQUE	Business key for the chat session
userId	TEXT	NOT NULL, FK -> USER.id	Reference to the owning user
messages	JSONB	NOT NULL	Array of chat message objects
files	JSONB	NOT NULL	Generated file structure (JSON tree)
createdAt	TIMESTAMP	DEFAULT NOW()	Chat session creation time

Entity: SESSION (Better Auth)

Attribute	Type	Constraint	Description
id	TEXT	PRIMARY KEY	Session token identifier
expiresAt	TIMEST	NOT NULL	Session expiry time

Attribute	Type	Constraint	Description
	AMP		
token	TEXT	NOT NULL, UNIQUE	Session authentication token
createdAt	TIMESTAMP	NOT NULL	Session creation time
updatedAt	TIMESTAMP	NOT NULL	Last session update
ipAddress	TEXT	NULLABLE	Client IP address
userAgent	TEXT	NULLABLE	Client user agent string
userId	TEXT	NOT NULL, FK -> USER.id	Reference to the authenticated user

Entity: ACCOUNT (OAuth Accounts)

Attribute	Type	Constraint	Description
id	TEXT	PRIMARY KEY	Account record identifier
accountId	TEXT	NOT NULL	Provider account ID
providerId	TEXT	NOT NULL	OAuth provider (google, github, etc.)
userId	TEXT	NOT NULL, FK -> USER.id	Reference to the local user
accessToken	TEXT	NULLABLE	OAuth access token

Attribute	Type	Constraint	Description
refreshToken	TEXT	NULLABLE	OAuth refresh token
idToken	TEXT	NULLABLE	OpenID Connect ID token
expiresAt	TIMESTAMP	NULLABLE	Token expiry time
createdAt	TIMESTAMP	NOT NULL	Account link creation time
updatedAt	TIMESTAMP	NOT NULL	Last update time

ERD Relationships

- USER to CHAT: One-to-Many. One user can have many chat sessions; each chat belongs to exactly one user (enforced by userId foreign key with CASCADE DELETE).
- USER to SESSION: One-to-Many. One user can have multiple active sessions (e.g., different browsers/devices).
- USER to ACCOUNT: One-to-Many. One user can link multiple OAuth accounts (e.g., both Google and GitHub).

Cloud IDE ERD — Container Entity

Cloud IDE does not use a persistent relational database. Instead, the 'entities' are Docker containers whose lifecycle is managed in memory by the Traefik management API. The conceptual entity is:

Attribute	Type	Source	Description

Attribute	Type	Source	Description
containerId	STRING	Docker-assigned	Unique Docker container ID (hex)
containerName	STRING	userId	Container name matches user ID
image	STRING	Config	Docker image: cloud-ide:latest
state	STRING	Docker runtime	running, stopped, exited
labels	MAP	Traefik config	traefik.enable, router rule, service port
port	INTEGER	Config	Internal port 9000 (WebSocket + REST)
userId	STRING	Client request	Identifier of the owning user

3.2 System Architecture

The system architecture for both VStack and Cloud IDE follows modern cloud-native principles: microservices separation of concerns, containerization for isolation, event-driven communication via WebSockets, and API-first design.

VStack — System Architecture

VStack is structured as a single Next.js 16 application using the App Router pattern. The architecture comprises the following layers:

Presentation Layer

The presentation layer consists of React Server Components (RSC) and Client Components. Server components handle initial data fetching and rendering, while client components manage interactive state (chat input, code preview toggling, payment UI). Tailwind CSS provides utility-based styling, with Radix UI primitives for accessible component behavior and Framer Motion for animations.

Application Layer (Server Actions)

Server Actions replace traditional REST API routes for most operations. Key server actions include:

- ProcessChat: Orchestrates the full AI generation cycle — validates authentication, checks token balance, calls Gemini AI chat session, calls Gemini AI code session, parses JSON output, updates the database.
- GetAiMessage: Calls the Gemini chatSession with the user prompt plus the React chat prompt template.
- GetTokens: Retrieves the current user's token balance from the database.
- UpdateChat: Upserts the chat record with updated messages and files.
- GetUser: Retrieves the authenticated user object via Better Auth session.

AI Layer

The AI layer uses the @google/generative-ai SDK. Two Gemini model instances are maintained:

- chatSession: A multi-turn conversation session (gemini-pro model) seeded with a React Chat Prompt that instructs the model to respond in a structured conversational format explaining the website it will generate.
- codeSession: A separate session (gemini-pro model) seeded with a React Code Prompt that instructs the model to output ONLY a valid JSON object representing the file tree of a React application, with no additional text.

Data Layer

The data layer uses Drizzle ORM with the Postgres.js driver connecting to a Supabase-hosted PostgreSQL instance. The schema is strictly typed in TypeScript. Database migrations are managed with drizzle-kit. The four core tables are: users, chats, sessions, and accounts.

Authentication Layer

Better Auth handles the complete authentication lifecycle: user registration, credential verification, session creation, session validation, and OAuth token exchange. Better Auth is configured with the Drizzle ORM adapter pointing to the Supabase database.

Payment Layer

PayPal integration uses the @paypal/react-paypal-js SDK. When a user's token balance reaches zero, they are prompted to purchase a token bundle. On successful PayPal transaction, the server action updates the user's token count in the database.

Cloud IDE — System Architecture

Cloud IDE is a TurboRepo monorepo with three packages: client (Next.js frontend), server (Node.js WebSocket server — runs inside Docker containers), and traefik (reverse proxy manager).

Client Package (Next.js Frontend)

The client is a Next.js application providing the IDE UI. It includes the following components:

- Titlebar: Displays the project name and container controls (start/stop buttons calling Docker management API routes).
- Sidebar: Navigation icons for Explorer, Search, Git, Extensions, and Settings (similar to VS Code layout).

- Explorer: File tree component that fetches the container's file structure from GET /files and renders it recursively. Clicking a file opens it in the editor.
- Editor (Monaco): Uses the Monaco Editor library to render file content with language detection and syntax highlighting. On save, content is POSTed to POST /files/:filePath.
- Tabsbar: Shows open file tabs; clicking switches between files.
- Xterm (Terminal): Renders an xterm.js terminal; receives terminal:data events from Socket.IO and sends keyboard input to POST /api/terminal.
- Bottombar: Status bar showing connection status, language, and line/column position.

Server Package (Node.js WebSocket Server — Inside Container)

The server runs inside each Docker container on port 9000. It is built with Express.js and Socket.IO. Key responsibilities:

- Spawns a node-pty pseudo-terminal running bash in the /user directory.
- Forwards terminal output (ptyProcess.onData) to all connected Socket.IO clients via terminal:data events.
- Accepts terminal input via POST /api/terminal and writes it to the pty process.
- Serves file tree via GET /files using a recursive async directory traversal.
- Serves file content via GET /files/:filePath.
- Accepts file saves via POST /files/:filePath.
- Watches the /user directory with chokidar and emits file-change events via Socket.IO on any filesystem modification.

Traefik Package (Reverse Proxy Manager)

The Traefik package contains two components:

- Traefik Service (Docker Compose): A Traefik v2.9 container configured to use Docker as its provider. It watches Docker labels and automatically creates routing rules. Port 80 is the main entrypoint.

- Management API (Express.js): Runs on port 8080 on the host. Uses Dockerode to interact with the Docker daemon via /var/run/docker.sock. Exposes three endpoints: POST /start (creates and starts a user container with Traefik labels), POST /running (checks if a user's container is active), and POST /stop (stops and removes the container).

Request Flow in Cloud IDE

When a user navigates to the Cloud IDE frontend and their container is running, the following routing occurs:

25. Browser sends request to userid.localhost.
26. Traefik receives the request on port 80.
27. Traefik matches the Host header to the router rule registered for that userId.
28. Traefik forwards the request to the container's port 9000.
29. The WebSocket server inside the container handles the request.
30. Response travels back through Traefik to the browser.

3.3 Database Requirements and User Interfaces

Database Requirements — VStack

VStack's database is a PostgreSQL instance hosted on Supabase. The database is accessed exclusively through Drizzle ORM, ensuring type-safety and preventing SQL injection. The following key design decisions were made:

- JSONB columns for messages and files: The messages column stores the full conversation history as a JSON array of {role, content} objects. The files column stores the generated file structure as a nested JSON object. JSONB provides efficient storage and querying of these semi-structured data types.
- Foreign key constraints: All relationships are enforced at the database level with appropriate ON DELETE CASCADE rules, ensuring referential integrity.
- Row-Level Security (RLS): Supabase RLS policies restrict access to chat rows so that users can only read and write their own records.

- Drizzle Kit for migrations: Schema changes are applied via Drizzle Kit's push command, which generates and applies SQL migration files.

Connection Configuration

Parameter	Value	Purpose
Database Provider	Supabase (managed PostgreSQL)	Hosting & connection pooling
Connection URL	Environment variable: DATABASE_URL	Supabase connection string
ORM	Drizzle ORM with postgres.js driver	Type-safe query building
Schema file	src/service/Database/schema.ts	Table definitions in TypeScript
Migrations directory	src/service/Database/migrations	SQL migration history
Dialect	PostgreSQL	Database dialect for Drizzle Kit

User Interface Design — VStack

The VStack UI is designed around three primary screens:

Home Screen

The home screen presents a centered layout with the VStack branding, an animated headline using the Cover component (text animation effect), a motivational subtitle, a large textarea for inputting website descriptions, and a grid of suggestion chips for quick-start prompts. A 'Stacks' section below shows template categories. The

navigation bar contains the app logo, a sign-in button (or avatar dropdown for authenticated users).

Chat / Generation Screen (/chat/[id])

The generation screen is split into two panels: a left panel containing the chat conversation (AI responses displayed as markdown-rendered text) and a right panel containing the generated code preview. The right panel has two tabs: Preview (Sandpack live preview) and Code (file tree + Monaco code editor). The chat input at the bottom allows iterative refinement of the generated code.

Tokens Screen (/tokens)

The tokens screen displays the user's current token balance and provides a PayPal payment button to purchase additional tokens. It shows a pricing breakdown of token packages and the expected number of AI generations each package supports.

User Interface Design — Cloud IDE

The Cloud IDE UI replicates the familiar layout of desktop IDEs like VS Code:

- Title Bar: Top bar with project name and session controls.
- Activity Bar / Sidebar: Left panel with icons for File Explorer, Search, and other panels.
- Editor Area: Center panel with Monaco Editor for the currently open file.
- Tab Bar: Row of tabs above the editor for each open file.
- Terminal Panel: Bottom panel with xterm.js terminal connected to container bash.
- Status Bar: Bottom strip showing file type, connection status, and line/column.

UI Component Library

Component	Library	Purpose

Component	Library	Purpose
Button, Input, Textarea	Radix UI / shadcn/ui	Accessible base components
Avatar	Radix UI Avatar	User profile images
Dropdown Menu	Radix UI Dropdown	Navigation menus and context menus
Dialog	Radix UI Dialog	Modal dialogs (sign-in, payment)
Tooltip	Radix UI Tooltip	Hover information labels
Toast Notifications	react-hot-toast	User feedback messages
Icons	Lucide React	SVG icon library
Animations	Framer Motion	Page transitions and micro-interactions
Particle Effects	@tsparticles	Background particle animations
Markdown Rendering	react-markdown	AI response text rendering
Code Preview	Sandpack (CodeSandbox)	In-browser React code execution
IDE Editor	Monaco Editor	Code editing in Cloud IDE
Terminal	xterm.js	Browser terminal emulation in Cloud IDE

3.4 Data Flow Diagram (DFD)

Data Flow Diagrams illustrate how data moves through the system. Level 0 (Context Diagram) shows the system as a single process with external entities. Level 1 diagrams expand the main process into sub-processes.

VStack — DFD Level 0 (Context Diagram)

External Entities: USER (provides website description, receives generated website code and preview), GEMINI AI (receives prompts, returns chat response and code JSON), SUPABASE DATABASE (receives and returns persistent data), and PAYPAL (processes token purchase transactions).

VStack — DFD Level 1

Process No.	Process Name	Input Data	Output Data
1.1	Authenticate User	Credentials / OAuth token	Session token / auth error
1.2	Manage Chat Session	User message + chat ID	Chat ID, stored session
1.3	Call AI Chat Session	User message + React Chat Prompt	Conversational AI response text
1.4	Call AI Code Session	User message + React Code Prompt	JSON file structure object
1.5	Parse & Store Generated Code	JSON file tree from AI	Stored files in CHAT table
1.6	Render Live Preview	Generated React files	Sandpack preview iFrame
1.7	Manage Token	User ID, generation	Updated token count

Process No.	Process Name	Input Data	Output Data
	Balance	event	
1.8	Process Payment	PayPal transaction, user ID	Token credit, receipt

Cloud IDE — DFD Level 0 (Context Diagram)

External Entities: USER (interacts with file explorer, editor, and terminal), DOCKER DAEMON (manages container lifecycle), and TRAEFIK (routes HTTP/WebSocket requests to the correct container).

Cloud IDE — DFD Level 1

Process No.	Process Name	Input Data	Output Data
1.1	Check Container Status	User ID	Running / Not running status
1.2	Start Docker Container	User ID, Docker image	Container ID, Traefik labels registered
1.3	Route Request via Traefik	HTTP request to userid.localhost	Forwarded request to container:9000
1.4	Serve File Tree	GET /files request	JSON file tree object
1.5	Read File Content	GET /files/:path request	File content string

Process No.	Process Name	Input Data	Output Data
1.6	Save File Content	POST /files/:path + content	200 OK / error
1.7	Handle Terminal I/O	POST /api/terminal + keystroke	pty output via Socket.IO
1.8	Watch Filesystem	chokidar file events	file-change Socket.IO events
1.9	Stop Container	POST /stop + user ID	Container stopped and removed

3.5 Data Dictionary

The Data Dictionary provides a formal description of all data elements used in the system.

VStack Data Dictionary

Data Element	Type	Length	Description	Source
userId	TEXT (UUID)	36 chars	Unique identifier for a user; auto-generated by Better Auth	System
userName	TEXT	Max 255	User's full display name	User input / OAuth profile
userEmail	TEXT	Max 255	User's email address; used for login and	User input / OAuth profile

Data Element	Type	Length	Description	Source
			notifications	
emailVerified	BOOLEAN	-	TRUE if the email has been verified via verification link	System
userImage	TEXT (URL)	Max 500	URL to user's profile picture from OAuth provider	OAuth provider
tokens	INTEGER	-	Number of AI generation tokens remaining; decremented on each AI call	System
chatId	TEXT	~8 chars	Random alphanumeric ID generated client-side for each chat session	Client (Math.random)
userMessage	TEXT	Max 5000	The natural language website description entered by the user	User input
aiChatResponse	TEXT	Variable	The conversational response from Gemini AI explaining the generated website	Gemini AI
codeJSON	JSONB	Vari	Structured JSON	Gemini AI

Data Element	Type	Length	Description	Source
		able	object representing the file tree of the generated React app	
fileName	TEXT	Max 255	Name of a file within the generated file structure (e.g., App.jsx)	AI output
fileContent	TEXT	Variable	Source code content of a generated file	AI output
sessionToken	TEXT	64 chars	Cryptographically secure session token managed by Better Auth	Better Auth
sessionExpiry	TIMESTAMP	-	Expiry timestamp of the user session (typically 30 days)	Better Auth
paypalOrderId	TEXT	Max 64	PayPal transaction order ID returned by PayPal on payment	PayPal

Cloud IDE Data Dictionary

Data Element	Type	Description	Source
userId	STRING	Identifier for the user, used as container name and Traefik routing key	Client request
containerId	STRING (SHA256 hex)	Unique Docker-assigned container identifier	Docker daemon
containerImage	STRING	Docker image name for the IDE container (cloud-ide:latest)	System config
containerState	STRING	Current container state: created, running, paused, exited	Docker daemon
traefikLabel	STRING	Docker label value for Traefik routing rule: Host(`userId.localhost`)	System
socketId	STRING	Socket.IO connection identifier for a connected client	Socket.IO
terminalData	STRING	Raw terminal output bytes from node-pty, forwarded to xterm.js	node-pty
terminalInput	STRING	Keystrokes from user's xterm.js terminal, sent to pty.write()	User input
filePath	STRING	Relative file path within the /user directory of the container	File system

Data Element	Type	Description	Source
fileContent	STRING	UTF-8 encoded content of a file inside the container	Container FS
fileTree	JSON Object	Nested JSON object representing the directory tree (dirs as objects, files as null)	Server traversal
fileChangeEvent	STRING	chokidar event type: add, change, unlink, addDir, unlinkDir	chokidar watcher
healthStatus	STRING	Response to GET /health endpoint: '200 OK'	Express server
userDir	STRING	Absolute path of the working directory inside the container: /home/server/user	System config

3.6 Table Design

The following SQL table definitions describe the complete VStack database schema as implemented with Drizzle ORM:

Table: users

Column	Data Type	Nullable	Default	Constraint
id	TEXT	NO	-	PRIMARY KEY
name	TEXT	NO	-	-

Column	Data Type	Nullable	Default	Constraint
email	TEXT	NO	-	UNIQUE
emailVerified	BOOLEAN	NO	FALSE	-
image	TEXT	YES	NULL	-
createdAt	TIMESTAMP	NO	NOW()	-
updatedAt	TIMESTAMP	NO	NOW()	-
tokens	INTEGER	YES	10	-

Table: chats

Column	Data Type	Nullable	Default	Constraint
id	SERIAL	NO	AUTO	PRIMARY KEY
chatId	TEXT	NO	-	UNIQUE
userId	TEXT	NO	-	FK -> users.id ON DELETE CASCADE
messages	JSONB	NO	-	-
files	JSONB	NO	-	-
createdAt	TIMESTAMP	YES	NOW()	-

Table: sessions

Column	Data Type	Nullable	Default	Constraint
id	TEXT	NO	-	PRIMARY KEY
expiresAt	TIMESTAMP	NO	-	-
token	TEXT	NO	-	UNIQUE
createdAt	TIMESTAMP	NO	-	-
updatedAt	TIMESTAMP	NO	-	-
ipAddress	TEXT	YES	NULL	-
userAgent	TEXT	YES	NULL	-
userId	TEXT	NO	-	FK -> users.id ON DELETE CASCADE

Table: accounts

Column	Data Type	Nullable	Default	Constraint
id	TEXT	NO	-	PRIMARY KEY
accountId	TEXT	NO	-	-

Column	Data Type	Nullable	Default	Constraint
providerId	TEXT	NO	-	-
userId	TEXT	NO	-	FK -> users.id ON DELETE CASCADE
accessToken	TEXT	YES	NULL	-
refreshToken	TEXT	YES	NULL	-
idToken	TEXT	YES	NULL	-
expiresAt	TIMESTAMP	YES	NULL	-
createdAt	TIMESTAMP	NO	-	-
updatedAt	TIMESTAMP	NO	-	-

3.7 Code Design

The code design section describes the architectural patterns, module structure, and key design decisions made in the development of both VStack and Cloud IDE.

VStack — Module Structure

Directory/File	Type	Responsibility
src/app/	Next.js App Router	Page routing and layout definitions

Directory/File	Type	Responsibility
src/app/page.tsx	React Server Component	Home page with prompt input and suggestions
src/app/chat/[id]/page.tsx	React Client Component	Chat and code generation screen
src/app/tokens/page.tsx	React Client Component	Token balance and PayPal payment screen
src/app/sign-in/page.tsx	React Client Component	Authentication screen
src/app/api/auth/[...all]/route.ts	Next.js API Route	Better Auth request handler
src/actions/GetAi.ts	Server Action	Gemini AI integration: chat and code sessions
src/actions/GetChat.ts	Server Action	Database CRUD operations for chat sessions
src/actions/GetUser.ts	Server Action	User authentication and token management
src/actions/payment.ts	Server Action	PayPal payment processing and token credit
src/service/Ai.ts	Service Module	Gemini AI client initialization and session config
src/service/Auth/auth.ts	Service Module	Better Auth configuration with Drizzle adapter
src/service/Database/schema.ts	Drizzle Schema	TypeScript database schema definitions

Directory/File	Type	Responsibility
src/components/ChatView.tsx	React Component	Chat conversation display and input
src/components/EditorView.tsx	React Component	Code editor (Monaco) and Sandpack preview
src/components/MainSidebar.tsx	React Component	Chat history sidebar
src/lib/Constant.ts	Constants Module	AI prompt templates and suggestion strings
src/lib/Types.ts	TypeScript Types	Shared type definitions (Message, FileStructure)
src/lib/Context.tsx	React Context	Global state for user message and template
auth-schema.ts	Schema File	Better Auth database table definitions
drizzle.config.ts	Config File	Drizzle Kit migration configuration

Cloud IDE — Module Structure

Directory/File	Type	Responsibility
apps/client/src/app/page.tsx	Next.js Page	Main IDE UI layout and container lifecycle management
apps/client/src/app/api/docker/start/	Next.js API Route	Calls management API to start user container
apps/client/src/app/api/docker/stop/	Next.js API Route	Calls management API to stop user container

Directory/File	Type	Responsibility
apps/client/src/app/api/docker/running/	Next.js API Route	Calls management API to check container status
apps/client/src/components/Explorer.tsx	React Component	Recursive file tree viewer with click-to-open
apps/client/src/components/Editor.tsx	React Component	Monaco Editor wrapper for file content editing
apps/client/src/components/Xterm.tsx	React Component	xterm.js terminal connected to Socket.IO
apps/client/src/components/Titlebar.tsx	React Component	App title bar with container start/stop controls
apps/client/src/components/Sidebar.tsx	React Component	VS Code-style activity bar with panel icons
apps/client/src/components/Tabsbar.tsx	React Component	File tab management for open files
apps/client/src/components/Bottombar.tsx	React Component	Status bar with language and line info
packages/server/src/server.ts	Node.js Server	WebSocket + Express server running inside container
packages/server/Dockerfile	Docker Config	Container image definition for the IDE environment
packages/traefik/src/index.ts	Management API	Dockerode-based container lifecycle management
packages/traefik/docker-compose.yml	Docker Compose	Traefik service and management API configuration

Directory/File	Type	Responsibility
turbo.json	TurboRepo Config	Build pipeline and task dependency configuration
pnpm-workspace.yaml	Workspace Config	pnpm monorepo workspace package definitions

Key Design Patterns

- Server Actions (VStack): Next.js Server Actions are used instead of traditional API routes for all database and AI operations. This eliminates the need for fetch() calls in client components and reduces boilerplate.
- Dual AI Sessions (VStack): Separating the chat session (for conversation) from the code session (for JSON output) ensures that the code generation is reliable and parseable without contamination from conversational text.
- Docker Label-Based Routing (Cloud IDE): Using Traefik labels attached to Docker containers eliminates the need for a separate service registry. Traefik dynamically discovers and configures routes as containers start and stop.
- Pseudo-Terminal Multiplexing (Cloud IDE): A single node-pty process is spawned per container and its output is broadcast to all Socket.IO connections to that container. This simplifies the architecture while supporting multiple browser windows.
- Recursive Directory Tree (Cloud IDE): The getFileListTree function recursively builds a nested JSON object where directories are objects (children) and files are null (leaf nodes), providing a simple yet complete filesystem representation.

3.8 Menu Screens

VStack Navigation Structure

The VStack application has a flat, intuitive navigation structure designed for minimal friction:

Screen / Route	Access Method	Description
Home (/)	Direct URL / Logo click	Landing page with prompt input, suggestions, and stack templates
Sign In (/sign-in)	Auto-redirect if unauthenticated	Email/password and OAuth sign-in screen
Chat (/chat/[id])	After submitting a prompt	AI generation screen with chat + code preview panels
Tokens (/tokens)	Avatar dropdown > 'Manage Tokens'	Token balance display and PayPal purchase UI
Previous Chats	Left sidebar (home screen)	List of past chat sessions; click to reload
Account Settings	Avatar dropdown > 'Settings'	User profile and account management
Sign Out	Avatar dropdown > 'Sign Out'	Terminates session and redirects to home

Cloud IDE Navigation Structure

UI Element	Location	Function

UI Element	Location	Function
Start Session Button	Title Bar	Calls /api/docker/start to create and launch user container
Stop Session Button	Title Bar	Calls /api/docker/stop to terminate user container
File Explorer Icon	Activity Bar (left)	Shows/hides the file tree panel
Search Icon	Activity Bar (left)	Opens file search (future enhancement)
File Items	Explorer Panel	Click to open file in Monaco Editor
Folder Items	Explorer Panel	Click to expand/collapse directory
Tab	Tab Bar (top)	Click to switch to open file; X button to close
Terminal Panel	Bottom area	Interactive bash terminal via xterm.js
Language Indicator	Status Bar (bottom right)	Shows current file language
Connection Status	Status Bar (bottom left)	Shows WebSocket connection state

3.9 Input Screens

VStack — Primary Input: AI Prompt

Field	Type	Validation	Description
Website Description	Textarea (multiline)	Non-empty string required	Natural language description of the website to generate

Field	Type	Validation	Description
Submit Button (Forward icon)	Icon Button	Hidden when textarea is empty	Submits the prompt to start AI generation
Suggestion Chips	Clickable text badges	N/A (pre-defined values)	One-click quick-start prompts for common website types

VStack — Sign-In Form

Field	Type	Validation	Description
Email	Input (email type)	Valid email format required	User's registered email address
Password	Input (password type)	Minimum 8 characters	User's account password
Sign In Button	Submit Button	Form must be valid	Submits credentials to Better Auth
Google Sign In	OAuth Button	N/A	Initiates Google OAuth flow
GitHub Sign In	OAuth Button	N/A	Initiates GitHub OAuth flow
Create Account Link	Anchor Link	N/A	Navigates to registration form

Cloud IDE — Terminal Input

Input Element	Type	Behavior
xterm.js Terminal	Browser pseudo-terminal	Captures all keyboard input and sends to container via Socket.IO
Resize Handle	Draggable divider	Adjusts terminal panel height; triggers xterm fit addon
File Content Editor	Monaco Editor textarea	Accepts code input with IntelliSense and syntax highlighting
Save Shortcut	Ctrl+S keyboard event	Triggers POST /files/:filePath with current editor content

3.10 Report Formats

Both VStack and Cloud IDE generate various forms of output that constitute the system's 'reports' in the software engineering sense:

VStack — Output Reports

Output / Report	Format	Generated By	Displayed In
AI Chat Response	Markdown text	Gemini AI (chat session)	ChatView component (react-markdown rendered)
Generated Website Preview	Live React application	Sandpack renderer	Preview tab (iframe)
Generated File Tree	Monaco Editor file	AI code session	EditorView file sidebar

Output / Report	Format	Generated By	Displayed In
	list	JSON	
Generated Source Code	Syntax-highlighted code	Monaco Editor	Code tab in EditorView
Token Balance	Numeric display	Database query	Tokens page and Navbar avatar dropdown
Chat History List	Scrollable sidebar list	Database query	MainSidebar component
Payment Receipt	PayPal transaction ID	PayPal SDK	Toast notification

Cloud IDE — Output Reports

Output / Report	Format	Generated By	Displayed In
File Tree	Nested JSON rendered as tree UI	Server GET /files	Explorer component
File Content	Syntax-highlighted source code	Monaco Editor + GET /files/:path	Editor component
Terminal Output	ANSI-colored text	node-pty + bash	xterm.js Terminal

Output / Report	Format	Generated By	Displayed In
	stream		
File Change Events	Real-time event notifications	chokidar + Socket.IO	Explorer (auto-refreshes)
Container Status	Running / Not Running indicator	GET /api/docker/running	Titlebar status display
Error Messages	Toast notifications	Catch blocks in API routes	react-hot-toast overlay

3.11 Test Procedures and Implementation

A comprehensive testing strategy was employed covering unit testing, integration testing, and system-level end-to-end testing.

Testing Strategy

- Unit Testing: Individual functions and components tested in isolation.
- Integration Testing: Testing interactions between modules (e.g., server action + database, API route + Docker).
- System Testing: End-to-end testing of complete user flows.
- Performance Testing: Testing response times and concurrent user handling.
- Security Testing: Verifying authentication guards, cross-user isolation, and input sanitization.

Unit Test Cases — VStack

Test ID	Component/ Function	Test Description	Expected Result	Status
UT-V-01	GetAiMessage()	Call with valid user prompt	Returns {status:200, content: string}	Pass
UT-V-02	GetAiMessage()	Call when AI is unreachable	Returns {status:400, content: error msg}	Pass
UT-V-03	GetTokens()	Authenticated user with 10 tokens	Returns {status:200, tokens: 10}	Pass
UT-V-04	GetTokens()	Unauthenticated call	Returns {status:400, error: 'Not authenticated'}	Pass
UT-V-05	UpdateChat()	Valid chatId, messages, files	Upserts successfully; returns updated record	Pass
UT-V-06	UpdateChat()	Invalid chatId format	Returns database error gracefully	Pass
UT-V-07	JSON.parse(codeText)	Valid JSON from Gemini code session	Parses to FileStructure object	Pass
UT-V-08	JSON.parse(codeText)	Malformed JSON from AI	Caught in catch block; 500 error	Pass

Test ID	Component/ Function	Test Description	Expected Result	Status
			returned	
UT-V-09	ChatView component	Render with no messages	Renders empty state without crash	Pass
UT-V-10	Home page textarea	Empty textarea submit	Forward button hidden; no submit occurs	Pass

Integration Test Cases — Cloud IDE

Test ID	Integration Point	Test Description	Expected Result	Status
IT-C-01	Management API + Docker	POST /start with valid userId	Container created, started, Traefik labels set	Pass
IT-C-02	Management API + Docker	POST /start with already-running container	New container started (old removed on init)	Pass
IT-C-03	Management API + Docker	POST /running with active container	Returns {running: true}	Pass
IT-C-04	Management API + Docker	POST /stop with running container	Container stopped and removed;	Pass

Test ID	Integration Point	Test Description	Expected Result	Status
			{status:'success'}	
IT-C-05	Traefik + Container	HTTP GET to userid.localhost	Request routed to container port 9000	Pass
IT-C-06	Socket.IO + node-pty	Connect socket, send terminal command	Command output received via terminal:data event	Pass
IT-C-07	Express + Filesystem	GET /files on populated /user dir	Correct nested JSON tree returned	Pass
IT-C-08	Express + Filesystem	POST /files/:path with content	File written; subsequent GET returns same content	Pass
IT-C-09	chokidar + Socket.IO	Create new file in /user directory	file-change event emitted to connected clients	Pass
IT-C-10	Client + API Routes	Click Start in Titlebar	Container started; file tree loaded; terminal connected	Pass

System Test Cases — End-to-End

Test ID	User Scenario	Steps	Expected Outcome	Status
ST-01	VStack: New user generates first website	1.Open app 2.Sign up 3.Enter prompt 4.Submit	AI response shown; live preview renders correctly	Pass
ST-02	VStack: Token exhaustion and purchase	1.Use all 10 tokens 2.Click purchase 3.Complete PayPal	Tokens credited; generation enabled again	Pass
ST-03	VStack: Reload previous chat	1.Log in 2.Click sidebar chat entry 3.View	Previous messages and code files restored	Pass
ST-04	Cloud IDE: Full session lifecycle	1.Open IDE 2.Click Start 3.Browse files 4.Edit file 5.Use terminal 6.Stop	All features work correctly end-to-end	Pass
ST-05	Cloud IDE: Multi-user isolation	1.Open IDE as user-A 2.Open IDE as user-B 3.Edit different files	Changes are isolated; no cross-container data leakage	Pass
ST-06	Security: Unauthenticated AI call (VStack)	1.Log out 2.Submit prompt from home	Redirected to sign-in; no AI call made	Pass

CHAPTER 4: USER MANUAL

4.1 User Manual

This chapter provides step-by-step instructions for using both VStack and Cloud IDE. The manual is intended for end users with basic web browsing proficiency.

Getting Started with VStack

Step 1: Accessing VStack

Open your web browser (Google Chrome 120+ or Mozilla Firefox 120+ recommended) and navigate to the VStack application URL. You will see the home screen with the headline 'Generate your Own Website' and an animated cover effect.

Step 2: Creating an Account

31. Click the 'Sign In' button in the top-right navigation bar.
32. On the sign-in page, choose your preferred method:

- Email/Password: Enter your email address and create a password (minimum 8 characters), then click 'Sign Up'.
- Google OAuth: Click 'Sign in with Google' and authorize the application.
- GitHub OAuth: Click 'Sign in with GitHub' and authorize the application.

33. Upon successful registration, you will be redirected to the home screen with your avatar visible in the navigation bar.
34. New accounts are provisioned with 10 AI generation tokens automatically.

Step 3: Generating Your First Website

35. In the large text area on the home screen, type a description of the website you want to create. Be as specific as possible for better results.

Example prompts:

- 'Create a personal portfolio website for a graphic designer with a dark theme, hero section with my name and role, skills section showing Photoshop, Illustrator, Figma, and a contact form.'
- 'Build a restaurant landing page with a menu section, photo gallery, reservation form, and footer with address and social links.'
- 'Generate a SaaS product landing page with a hero, features grid, pricing table with three tiers, testimonials, and a call-to-action.'

36. Alternatively, click one of the pre-written suggestion chips below the text area to use a ready-made prompt.
37. Click the Forward (arrow) button that appears when text is entered, or press Enter, to start generation.

Step 4: Viewing and Using the Generated Website

38. The chat screen opens and displays the AI's conversational description of the generated website.
39. On the right panel, the Preview tab shows a live, interactive rendering of your generated website.
40. Click the Code tab to see the generated file structure. Click any file to open it in the code editor.
41. You can directly edit the code in the editor. Changes are reflected in the preview when you switch tabs.
42. Type a follow-up message in the chat input (e.g., 'Change the color scheme to purple and white') to refine the generated website.

Step 5: Managing Tokens

43. Each AI generation (each time you send a prompt) consumes one token.
44. Your token balance is visible in the avatar dropdown menu.
45. When your tokens run out, navigate to the Tokens page via the avatar dropdown.

46. Click the PayPal button, log in to your PayPal account, and complete the purchase.
47. Tokens are credited to your account immediately upon successful payment.

Getting Started with Cloud IDE

Step 1: Prerequisites

Ensure that the Cloud IDE server infrastructure is running. This requires:

- Docker Engine installed and running on the host server.
- Traefik and the Management API started via docker-compose up in the packages/traefik directory.
- The Cloud IDE Docker image (cloud-ide:latest) built and available locally.

Step 2: Opening the Cloud IDE

48. Open your web browser and navigate to the Cloud IDE client URL (typically http://localhost:3000 for local development).
49. The IDE interface loads with the VS Code-style layout: activity bar on the left, editor in the center, and terminal at the bottom.
50. If no container is running, the editor and terminal areas will show a 'not connected' state.

Step 3: Starting a Session

51. Click the Start button in the title bar (usually labeled 'Start Session' or shown as a play icon).
52. The system calls the Docker management API to create and start your dedicated container.
53. Within a few seconds, the container starts and Traefik registers the routing rule.
54. The file explorer populates with the initial file tree of the container's /user directory.

55. The terminal connects to the container's bash shell.

Step 4: Using the File Explorer

56. The left panel shows the file tree. Click folder names to expand or collapse them.
57. Click a file name to open it in the Monaco Editor in the center panel.
58. The file opens in a new tab in the tab bar.
59. Multiple files can be open simultaneously; switch between them by clicking their tabs.

Step 5: Editing Code

60. When a file is open in Monaco Editor, you can type directly to edit it.
61. Monaco provides syntax highlighting, auto-indentation, bracket matching, and basic IntelliSense for JavaScript, TypeScript, Python, HTML, CSS, and more.
62. Save your changes with Ctrl+S (Windows/Linux) or Cmd+S (Mac).
63. The saved content is written to the file inside your Docker container.

Step 6: Using the Terminal

64. The terminal panel at the bottom is a fully functional bash terminal running inside your Docker container.
65. Click inside the terminal and start typing commands.
66. Commands execute in the /user directory of your container by default.
67. You can install packages, run scripts, compile code, and perform any Linux bash operations.
68. The terminal supports all standard ANSI escape sequences for colored output.

Step 7: Stopping a Session

69. When you are done with your session, click the Stop button in the title bar.
70. The container is stopped and removed from Docker.

71. Note: Any files saved in the container will be lost unless you have configured a persistent Docker volume.

4.2 Operations Manual / Menu Explanation

VStack Operations Manual

Menu / Control	Location	Operation	Notes
Prompt Textarea	Home screen center	Type website description	Forward button appears when non-empty
Forward Button	Home screen, right of textarea	Submit prompt and begin AI generation	Hidden when textarea is empty
Suggestion Chips	Home screen, below textarea	Click to use pre-written prompt	8-10 common website types available
Stack Templates	Home screen, below suggestions	Click to use a framework template	Feature in development; shows 'coming soon' toast
VStack Logo / Home Link	Navigation bar, left	Navigate to home screen	Available from all pages
Sign In Button	Navigation bar, right (unauthenticated)	Navigate to sign-in page	Shown only when not logged in

Menu / Control	Location	Operation	Notes
User Avatar	Navigation bar, right (authenticated)	Opens dropdown menu	Shows user name and email
Token Balance (in dropdown)	Avatar dropdown	Shows current token count	Read-only display
Manage Tokens (dropdown item)	Avatar dropdown	Navigate to /tokens page	Opens token purchase UI
Settings (dropdown item)	Avatar dropdown	Navigate to account settings	Profile management
Sign Out (dropdown item)	Avatar dropdown	Log out and redirect to home	Clears session cookie
Sidebar Chat List	Left panel (home screen)	Click entry to reload that chat	Shows chat ID; most recent first
Preview Tab	Right panel (chat screen)	Switch to live website preview	Sandpack iframe rendering
Code Tab	Right panel (chat screen)	Switch to file tree and Monaco Editor	Shows all generated files
File List (in Code tab)	Right panel, code	Click file name to open in editor	Files from AI-generated tree

Menu / Control	Location	Operation	Notes
	tab		
Chat Input	Bottom of chat screen	Type follow-up prompt for iteration	Each submission uses one token

Cloud IDE Operations Manual

Control	Location	Operation	Behavior
Start Session	Title Bar	Click to start user container	Calls /api/docker/start; waits for container
Stop Session	Title Bar	Click to stop user container	Calls /api/docker/stop; clears IDE state
File Explorer Icon	Activity Bar	Toggle file explorer panel	Shows/hides left panel
Refresh File Tree	Explorer panel	Re-fetch file tree from container	Updates on file-change events (automatic)
Expand Folder	Explorer panel	Click folder name	Toggles directory open/closed
Open File	Explorer panel	Click file name	Opens file in Monaco Editor; adds tab
Close Tab	Tab Bar	Click X on tab	Closes file; reverts to previous tab
Switch Tab	Tab Bar	Click tab label	Switches active editor file

Control	Location	Operation	Behavior
Save File	Monaco Editor	Ctrl+S / Cmd+S	POSTs current content to server
Resize Terminal	Horizontal divider	Drag up/down	Adjusts terminal/editor split
Clear Terminal	Terminal area	Type 'clear' or Ctrl+L	Clears xterm.js terminal output
Copy Terminal Text	Terminal area	Select text, Ctrl+C	Copies selected terminal text to clipboard

4.3 Forms and Report Specifications

VStack Form Specifications

Form Name	Fields	Validation Rules	Submit Action
Sign Up Form	Name, Email, Password, Confirm Password	Name: required; Email: valid format; Password: min 8 chars; Confirm matches	POST to Better Auth /sign-up endpoint
Sign In Form	Email, Password	Email: valid format; Password: required	POST to Better Auth /sign-in endpoint
AI Prompt Input	Website description (textarea)	Non-empty; max 5000 characters	Calls ProcessChat server action

Form Name	Fields	Validation Rules	Submit Action
Chat Follow-up Input	Follow-up message (textarea)	Non-empty	Calls ProcessChat with updated messages array
Token Purchase	PayPal payment (SDK-managed)	PayPal handles all payment validation	PayPal SDK triggers onApprove callback

Cloud IDE Form Specifications

Form Name	Input Method	Validation	Action
Terminal Input	xterm.js keyboard capture	Any printable character / control sequence	POSTed to /api/terminal; written to node-pty
File Editor Input	Monaco Editor key events	Any valid UTF-8 text	Ctrl+S triggers POST /files/:path
Start Session	Button click	No input fields	Calls /api/docker/start with userId
Stop Session	Button click	No input fields	Calls /api/docker/stop with userId

Report Specifications

Report Name	Data Source	Format	Update Frequency

Report Name	Data Source	Format	Update Frequency
AI Chat Response	Gemini AI chat session	Markdown text (react-markdown)	On each user prompt submission
Generated Website Preview	Sandpack + generated files	Live iFrame rendering	On code generation / file edit + preview switch
Generated File Tree	AI code JSON output	File list with click-to-open	On each new generation
Token Balance	Supabase users.tokens	Integer (e.g., '7 tokens remaining')	On page load; after payment; after generation
Chat History	Supabase chats table	Scrollable list of chat IDs	On home screen load
Container File Tree	Server GET /files	Recursive tree component	On connection; on file-change Socket.IO event
Terminal Output	node-pty stdout/stderr	ANSI text stream in xterm.js	Real-time streaming
Container Status	Docker management API	Running / Stopped status indicator	On page load; after start/stop actions

DRAWBACKS AND LIMITATIONS

Despite the innovative nature of both VStack and Cloud IDE, several drawbacks and limitations exist in the current implementation. These are acknowledged transparently to ensure academic integrity and to inform future development.

VStack — Drawbacks and Limitations

1. AI Output Reliability

Gemini AI, like all large language models, does not guarantee perfectly valid or parseable output every time. If the AI returns malformed JSON in the code session, the parsing step (JSON.parse) will throw an exception, resulting in a 500 error returned to the user. The current error handling catches this gracefully, but no automatic retry mechanism is implemented. Some prompts produce suboptimal results requiring multiple iteration prompts.

2. Token-Based Rate Limiting is Not Abuse-Proof

The current token system deducts one token per generation attempt. However, if a server action fails before the token decrement step (e.g., during AI call), the token is not deducted, which is the correct behavior. Conversely, if the token decrement succeeds but the AI call subsequently fails, the user loses a token for no output. A transactional approach (deduct only on confirmed success) would require additional database transaction management.

3. Generated Code Quality Depends on Prompt Engineering

The quality of generated React code is heavily dependent on the prompt templates defined in the React_Chat_Prompt and React_Code_Prompt constants. Poorly engineered prompts produce inconsistent or non-functional code. The current prompts produce good results for standard use cases but may struggle with complex, multi-page applications.

4. No Export / Deployment Feature

VStack currently allows users to view and edit generated code within the Sandpack sandbox but does not provide a mechanism to download the project as a ZIP file or deploy it directly to platforms such as Vercel or Netlify. Users who want to use the generated code must manually copy it from the editor.

5. No Real-time Collaborative Editing

VStack's code editor (Sandpack) does not support multi-user real-time collaboration. Only one user can edit a chat session's generated code at a time.

Cloud IDE — Drawbacks and Limitations

1. No Persistent Storage Across Sessions

When a user stops their container (or the container is removed), all files created or modified within the container are lost. The current implementation does not mount a persistent Docker volume to preserve user data between sessions. This severely limits the practical utility for ongoing projects.

2. No Authentication

Cloud IDE currently has no authentication system. Any user who can access the frontend can start a container and interact with the system. This is a significant

security gap that makes the current implementation unsuitable for production deployment on public networks without additional security layers (e.g., reverse proxy authentication, VPN).

3. Single node-pty Instance

The current server implementation spawns a single node-pty process per container. If multiple browser windows connect to the same container, they all share the same terminal session (all seeing the same terminal I/O). A proper implementation would create a new pty session per Socket.IO connection.

4. Resource Exhaustion Risk

Without resource limits on Docker containers, a malicious or careless user could consume excessive CPU or memory on the host, affecting all other users. CPU and memory limits (--cpus, --memory flags in Docker) should be applied to each container in production.

5. Limited to localhost Routing

The Traefik configuration in the docker-compose.yml file uses .localhost subdomains for routing (e.g., userid.localhost). While this works in local development, production deployment requires a real domain with wildcard DNS (*.yourdomain.com) and TLS certificate configuration, which adds significant infrastructure complexity.

6. No Git Integration

The Cloud IDE does not provide any version control features. Users cannot commit, push, or pull code from Git repositories within the IDE. This is a fundamental feature gap compared to commercial cloud IDEs like GitHub Codespaces.

PROPOSED ENHANCEMENTS

Based on the identified drawbacks, user feedback analysis, and emerging industry trends, the following enhancements are proposed for future iterations of the VStack and Cloud IDE platforms:

VStack — Proposed Enhancements

1. Project Export and One-Click Deployment

Priority: High. Implement a 'Download Project' button that packages all generated files as a ZIP archive for direct download. Additionally, integrate with the Vercel Deployment API and Netlify API to enable one-click deployment of generated websites to production hosting, transforming VStack from a prototype tool to a full deployment platform.

2. Multi-File Project Templates (Stacks)

Priority: High. Complete the 'Stacks' feature visible on the home screen. Allow users to select from pre-built project templates (e.g., Next.js full-stack, React + Express, Vue.js SPA) which provide a starting structure that the AI then customizes based on the user's prompt, enabling generation of more complex, multi-page applications.

3. AI Provider Abstraction Layer

Priority: Medium. Implement a provider-agnostic AI abstraction layer that supports multiple LLM providers (Google Gemini, OpenAI GPT-4, Anthropic Claude, Mistral). Users could select their preferred model, and the system would route to the appropriate API, reducing vendor lock-in and allowing quality comparison.

4. Real-Time Collaborative Editing

Priority: Medium. Integrate Yjs (a CRDT-based collaborative editing library) with Monaco Editor to enable multiple users to simultaneously edit the same generated codebase with cursor presence indicators and conflict-free merging, similar to Google Docs for code.

5. Automatic Retry on AI Parse Failure

Priority: Medium. Implement an automatic retry mechanism in the ProcessChat server action. If JSON.parse fails on the AI code output, the system should automatically re-call the code session with an amended prompt that emphasizes 'return ONLY valid JSON' and attempt parsing up to three times before returning an error.

6. Visual Website Builder Integration

Priority: Low. Add a drag-and-drop visual editing layer on top of the Sandpack preview that allows non-technical users to make visual changes (move elements, change colors, resize sections) which are then reflected back as code changes in the editor, bridging the gap between visual and code-based editing.

Cloud IDE — Proposed Enhancements

1. Persistent Volume Storage

Priority: Critical. Mount a user-specific Docker volume or bind mount to each container so that files in the /user directory persist across container restarts and session terminations. Implement a volume management system that creates, mounts, and backs up user volumes.

2. User Authentication and Authorization

Priority: Critical. Integrate Better Auth or NextAuth into the Cloud IDE frontend to require user authentication before starting containers. The userId should be derived from the authenticated session rather than a client-provided value, preventing container hijacking.

3. Git Integration

Priority: High. Implement Git commands in the IDE by adding a Git panel (similar to VS Code's Source Control panel). Users should be able to: git init, git clone from GitHub/GitLab, commit changes, push to remote, and view diff of changed files. The isomorphic-git library can provide Git operations without requiring Git installed on the host.

4. Docker Resource Limits

Priority: High. Add CPU and memory limits to container creation in the management API. Each container should be limited to a fair resource allocation (e.g., 0.5 CPU, 512MB RAM for free tier; 2 CPU, 2GB RAM for premium tier) to prevent resource exhaustion and enable fair multi-tenancy.

5. Production Domain and TLS

Priority: High. Configure Traefik with a wildcard DNS entry (*.cloudide.yourdomain.com) and Let's Encrypt ACME TLS certificate challenge to enable production HTTPS deployment. The acme.json configuration in the traefik package already includes the certificate resolver configuration; it requires domain and DNS setup.

6. VStack + Cloud IDE Integration

Priority: High. The most impactful proposed enhancement is the deep integration of VStack and Cloud IDE. When a user generates a website in VStack, they should be

able to click 'Open in Cloud IDE' which: creates a container, populates the /user directory with the generated files, and opens the Cloud IDE connected to that container. This creates the complete AI-generate-then-professional-edit workflow.

7. Multiple Terminal Sessions

Priority: Medium. Replace the single shared node-pty with a session-multiplexed terminal manager. Each Socket.IO connection should get its own pty instance (or a tmux pane), enabling independent terminal sessions per browser window and supporting split-terminal views.

CONCLUSIONS

This Industrial Project has successfully delivered two innovative, interconnected web development platforms — VStack, an AI-powered website generator, and Cloud IDE, a Docker-containerized cloud-based Integrated Development Environment — that together represent a significant contribution to the field of developer tooling and cloud computing.

The development of VStack demonstrates the practical viability of integrating large language model AI (specifically Google Gemini AI) into the web development workflow to dramatically reduce the time-to-prototype for web applications. By accepting natural language descriptions and generating complete, functional React codebases with live previews, VStack lowers the barrier to web development for non-experts while providing a powerful iteration tool for experienced developers. The integration of Supabase for data persistence, Drizzle ORM for type-safe database access, Better Auth for secure authentication, and PayPal for monetization creates a production-ready, commercially viable platform.

The development of Cloud IDE demonstrates the power of Docker containerization and WebSocket technology in delivering rich, environment-consistent development experiences entirely through the browser. By giving each user a dedicated, isolated Docker container with a full bash environment, Monaco Editor, and xterm.js terminal — all dynamically routed by Traefik — Cloud IDE eliminates the 'works on my machine' problem and makes professional development tools accessible without any local installation.

From a technical standpoint, this project has provided deep, hands-on experience with a broad range of modern technologies: Next.js App Router with Server Actions, TurboRepo monorepo management, Docker and Dockerode for container automation, Traefik as a dynamic reverse proxy, node-pty for pseudo-terminal management, Socket.IO for real-time WebSocket communication, and multiple database and authentication libraries. The project also required substantial prompt engineering skill to produce reliable, parseable AI outputs from Gemini.

The identified limitations — particularly the lack of persistent storage in Cloud IDE and the absence of authentication — are acknowledged as areas for immediate future work. The proposed enhancements, especially the integration of VStack and Cloud IDE into a unified AI-to-deployment pipeline, represent a compelling roadmap for a commercially significant product.

In conclusion, this project has met and exceeded its stated objectives: it has delivered two functional, innovative, open-source platforms; demonstrated the integration of AI, cloud, and containerization technologies; and produced a comprehensive technical report documenting the design, implementation, and testing process. The platforms developed in this project are directly aligned with current industry trends, including AI-assisted development, cloud-native architecture, and the growing importance of reproducible development environments — making this project not only academically rigorous but also industrially relevant.

The knowledge gained during this project — spanning full-stack TypeScript development, cloud infrastructure, DevOps principles, AI integration, and software engineering methodology — provides an excellent foundation for a career in modern software engineering or further research in AI-powered developer tools.

BIBLIOGRAPHY

The following references were consulted during the research, design, and development phases of this project:

Official Documentation

72. Next.js Documentation. (2024). App Router: Server Actions and Mutations. Vercel, Inc. Retrieved from https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations
73. Google. (2024). Gemini API Documentation: Generative AI for Developers. Google LLC. Retrieved from https://ai.google.dev/docs
74. Docker Inc. (2024). Docker Engine Documentation. Docker, Inc. Retrieved from https://docs.docker.com
75. Traefik Labs. (2024). Traefik Proxy Documentation v2.9. Traefik Labs. Retrieved from https://doc.traefik.io/traefik
76. Supabase Inc. (2024). Supabase Documentation: PostgreSQL Database Platform. Supabase, Inc. Retrieved from https://supabase.com/docs
77. Drizzle Team. (2024). Drizzle ORM Documentation. Drizzle Team. Retrieved from https://orm.drizzle.team/docs
78. Better Auth. (2024). Better Auth Documentation: Authentication for TypeScript. Retrieved from https://www.better-auth.com/docs
79. Socket.IO Team. (2024). Socket.IO Documentation v4.x. Retrieved from https://socket.io/docs/v4
80. Microsoft. (2024). Monaco Editor Documentation. Microsoft Corporation. Retrieved from https://microsoft.github.io/monaco-editor
81. xterm.js Contributors. (2024). xterm.js Documentation. Retrieved from https://xtermjs.org

82. CodeSandbox. (2024). Sandpack Documentation: In-Browser Code Bundler. CodeSandbox B.V. Retrieved from https://sandpack.codesandbox.io/docs
83. TurboRepo. (2024). Turborepo Handbook. Vercel, Inc. Retrieved from https://turbo.build/repo/docs

Books

84. Wieruch, R. (2023). The Road to Next.js: Building Modern React Applications. Self-published. Retrieved from https://www.roadtonextjs.com
85. Kinsman, C. (2022). Docker Deep Dive. Pluralsight LLC.
86. Brown, E. (2022). Web Development with Node and Express: Leveraging the JavaScript Stack (3rd ed.). O'Reilly Media.
87. Belcic, I., & Shafer, M. (2023). TypeScript Handbook. Microsoft Corporation. Retrieved from https://www.typescriptlang.org/docs/handbook

Research Papers and Articles

88. Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374
89. Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, 2014(239).
90. Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455. IETF. Retrieved from https://tools.ietf.org/html/rfc6455
91. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
92. Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.

Online Resources

93. Dockerode npm Package. (2024). Retrieved from https://www.npmjs.com/package/dockerode

94. node-pty npm Package. (2024). Retrieved from https://www.npmjs.com/package/node-pty
95. chokidar npm Package. (2024). Retrieved from https://www.npmjs.com/package/chokidar
96. PayPal Developer Documentation. (2024). PayPal REST APIs. PayPal Holdings Inc. Retrieved from https://developer.paypal.com/docs
97. MDN Web Docs. (2024). WebSockets API. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
98. Radix UI Documentation. (2024). Unstyled, Accessible Components for React. WorkOS. Retrieved from https://www.radix-ui.com

ANNEXURE 1: INPUT FORMS WITH DATA

Sample 1: VStack AI Prompt Input Form

Field	Sample Input Data
Website Description	Create a personal portfolio website for a software developer named Rohit. Include a dark-themed hero section with name, role as 'Full Stack Developer', and a CTA button. Add a skills section with React, Node.js, Next.js, Docker, and TypeScript. Include a projects section with three project cards showing project name, description, and GitHub link. Add a contact form with name, email, and message fields. Use a modern, minimal dark theme with blue accents.
User Authentication	Email: rohit.dev@example.com | Password: ********
Chat Session ID	x7k2m9p3 (auto-generated)
Token Balance Before	10 tokens
Token Balance After	9 tokens (1 deducted)
Timestamp	2025-03-20 14:32:17 UTC

Sample 2: Cloud IDE Container Start Request

API Parameter	Sample Value
Endpoint	POST http://localhost:8080/start

API Parameter	Sample Value
Request Body (JSON)	{ "userId": "user-rohit-abc123" }
Docker Image	cloud-ide:latest
Container Name	user-rohit-abc123
Traefik Label: enable	traefik.enable=true
Traefik Label: router rule	traefik.http.routers.user-rohit-abc123.rule=Host(`user-rohit-abc123.localhost`)
Traefik Label: entrypoint	traefik.http.routers.user-rohit-abc123.entrypoints=web
Traefik Label: service port	traefik.http.services.user-rohit-abc123.loadbalancer.server.port=9000
Response Body (JSON)	{ "status": "success", "container": "/user-rohit-abc123.localhost" }
Response Status	200 OK

Sample 3: VStack Sign-Up Form

Field Name	Input Type	Sample Value	Validation Result
Full Name	TEXT	Rohit Kumar	Valid
Email Address	EMAIL	rohit.kumar@ecb.ac.in	Valid — unique email
Password	PASSWOR	SecurePass@2025	Valid — 14 chars, mixed

Field Name	Input Type	Sample Value	Validation Result
	D		case, special char
Confirm Password	PASSWORD	SecurePass@2025	Valid — matches password
OAuth Provider (alternate)	BUTTON	Google / GitHub	N/A — no local password needed

Sample 4: File Save Request (Cloud IDE)

Request Parameter	Value
Method	POST
URL	http://user-rohit-abc123.localhost/files/App.jsx
Content-Type Header	application/json
Request Body	{ "content": "import React from 'react';\n\nexport default function App() {\n return Hello Cloud IDE! ;\n}" }
Response Status	200 OK
Response Body	File saved successfully
File Path on Container	/home/server/user/App.jsx

ANNEXURE 2: OUTPUT REPORTS WITH DATA

Output 1: Gemini AI Chat Response (Sample)

The following is a representative sample of the conversational AI response generated by Gemini in response to the portfolio website prompt:

AI Response: I've created a stunning personal portfolio for Rohit! The design features a sleek dark theme with electric blue accents that scream 'professional developer'. Here's what's included:

- Hero Section: A full-screen landing with your name, 'Full Stack Developer' title, a pulsating CTA button ('View My Work'), and a particle animation background.
- Skills Section: An animated grid of skill badges covering React, Node.js, Next.js, Docker, and TypeScript — each with their official icons.
- Projects Section: Three project cards with hover effects showing project screenshots, descriptions, tech stack tags, and live/GitHub links.
- Contact Form: A clean form with name, email, message inputs and validation feedback, connected to EmailJS for real email delivery.
- Responsive Design: Fully responsive across mobile, tablet, and desktop using Tailwind CSS breakpoints.

Output 2: Generated File Structure (JSON Sample)

File Name	Description	Approximate Size
index.jsx	Main App component with routing between sections	~120 lines
Hero.jsx	Hero section with name, role,	~80 lines

File Name	Description	Approximate Size
	CTA, and particle effect	
Skills.jsx	Skills grid with animated badge components	~60 lines
Projects.jsx	Projects cards with hover effects and links	~90 lines
Contact.jsx	Contact form with validation and EmailJS integration	~70 lines
Navbar.jsx	Sticky navigation bar with smooth scroll links	~50 lines
Footer.jsx	Footer with social media icons and copyright	~30 lines
styles.css	Global CSS and Tailwind configuration	~40 lines
package.json	Dependencies: react, react-dom, tailwindcss, lucide-react	~25 lines

Output 3: Docker Container Status Report

Field	Value
Container ID	a3f7b2c9d1e4 (Docker short ID)
Container Name	/user-rohit-abc123
Image	cloud-ide:latest
Status	Up 23 minutes

Field	Value
Ports	0.0.0.0:9000->9000/tcp (internal)
Network Mode	bridge
Traefik Routes Registered	user-rohit-abc123.localhost -> port 9000
Memory Usage	87.3 MB / 512 MB
CPU Usage	0.02% (idle)
Uptime	23 minutes 14 seconds

Output 4: File Tree Structure (Cloud IDE Sample Output)

Path	Type	Size
user/	Directory	-
user/index.html	File	1.2 KB
user/style.css	File	0.8 KB
user/app.js	File	3.4 KB
user/components/	Directory	-
user/components/Header.jsx	File	2.1 KB
user/components/Footer.jsx	File	1.0 KB
user/assets/	Directory	-

Path	Type	Size
user/assets/logo.png	File	45.2 KB
user/package.json	File	0.6 KB
user/README.md	File	0.4 KB

ANNEXURE 3: SAMPLE CODE SNIPPETS

The following sample code snippets illustrate key implementation patterns used in both VStack and Cloud IDE. These are representative excerpts to demonstrate code quality and architectural decisions. Full source code is available in the project ZIP files.

Snippet 1: VStack — ProcessChat Server Action (Key Logic)

Aspect	Description
File	src/actions/GetAi.ts
Pattern	Next.js Server Action with dual AI session calls
Key Logic	1. Validate auth 2. Check tokens 3. Call Gemini chat 4. Call Gemini code 5. Parse JSON 6. Update DB 7. Return to client
Error Handling	Try/catch wraps all operations; specific error codes returned
Token Management	GetTokens() called first; token decrement in UpdateChat

Snippet 2: Cloud IDE — Container Lifecycle (Management API)

Aspect	Description
File	packages/traefik/src/index.ts
Pattern	Dockerode API for container lifecycle management
POST /start	Check image exists > pull if needed > createContainer with

Aspect	Description
	Traefik labels > container.start()
POST /running	listContainers({ all: false }) > filter by name > return boolean
POST /stop	getContainer(userId) > container.stop() > container.remove()
Traefik Integration	Labels set on container creation; Traefik auto-discovers and creates routes

Snippet 3: Cloud IDE — WebSocket Server File Operations

Aspect	Description
File	packages/server/src/server.ts
Pattern	Express + Socket.IO + node-pty + chokidar
Terminal I/O	node-pty.onData() -> io.emit('terminal:data'); POST /api/terminal -> ptyProcess.write()
File Tree	Recursive async getFileListTree() builds nested JSON object
File Read/Write	fs.readFile() / fs.writeFile() with UTF-8 encoding
File Watching	chokidar.watch(USER_DIR) on all events -> io.emit('file-change', {event, path})

Snippet 4: VStack — Drizzle ORM Schema Definition

Table	Key Columns	ORM Pattern	Notable
users	id: text PK, email: text unique, tokens: integer(10)	pgTable('users', {...})	tokens default 10 for new users
chats	chatId: text unique, userId: FK, messages: jsonb, files: jsonb	pgTable('chats', {...})	JSONB for flexible message storage
sessions	token: text unique, userId: FK, expiresAt: timestamp	pgTable('session', {...})	Better Auth managed table
accounts	accountId, providerId, userId: FK, accessToken	pgTable('account', {...})	OAuth provider account links

Snippet 5: Docker Compose — Traefik Configuration

Service	Key Configuration	Purpose
traefik	image: traefik:v2.9, --providers.docker=true	Main reverse proxy
traefik	--entrypoints.web.address=:80	HTTP entrypoint on port 80
traefik	volumes: /var/run/docker.sock	Docker socket for label discovery

Service	Key Configuration	Purpose
reverse-proxy-app	build: ./Dockerfile, ports: 8080:8080	Management API for container CRUD
reverse-proxy-app	volumes: /var/run/docker.sock	Dockerode access to Docker daemon
network_mode: bridge	Applied to both services	Enables cross-service communication

Snippet 6: Cloud IDE — Dockerfile for Container Image

Instruction	Value	Purpose
FROM	node:22-alpine	Lightweight Node.js base image
RUN apk add	curl bash python3 make g++ build-base	Tools needed for node-pty native compilation
RUN ln -sf	/usr/bin/python3 /usr/bin/python	Python symlink for build tools
WORKDIR	/home/server/	Set working directory for app
COPY	. .	Copy all server source files
RUN npm install	-	Install Node.js dependencies including node-pty
EXPOSE	9000	Expose WebSocket server port
CMD	["npm", "run", "dev"]	Start the WebSocket server

--- END OF REPORT ---

ANNEXURE 4

Complete Source Code Documentation

AI-Powered Web Development Platform with Cloud IDE

VStack · Cloud IDE

Aspect	Detail / Explanation
Project	AI-Powered Web Development Platform with Cloud IDE
Module A	VStack — AI Website Generator (Next.js + Gemini AI + Supabase)
Module B	Cloud IDE — Containerised Browser IDE (Docker + Traefik + Socket.IO)
Language	TypeScript (strict) throughout both modules
Framework	Next.js 16 App Router (VStack) · Node.js + Express (Cloud IDE server)
Database	PostgreSQL via Supabase + Drizzle ORM (VStack)
Auth	Better Auth with Drizzle adapter + GitHub/Google OAuth (VStack)
Containers	Docker Engine 24 + Dockerode + Traefik v2.9 (Cloud IDE)
Terminal	node-pty (pseudo-terminal) + xterm.js + Socket.IO (Cloud IDE)
Editor	Monaco Editor (VS Code engine) — Cloud IDE frontend
Payments	PayPal REST SDK via @paypal/react-paypal-js (VStack)
Prepared by	Department of Computer Application, Engineering College Bikaner

— All code shown is the actual project source code —

HOW TO READ THIS DOCUMENT

This Annexure contains the complete, lightly formatted source code for every significant file in both the VStack and Cloud IDE projects. Each section includes: (a) the file path within the project, (b) the programming language, (c) an annotation table explaining key design decisions, and (d) the full source code in a dark monospaced code block identical in style to the VS Code editor used during development.

Line numbers are shown on the left of each code block. Inline comments (lines starting with // or prefixed with ——) have been added to explain non-obvious logic without modifying the functional code. The document is organised in dependency order: database schema first, then server actions, then React components, then infrastructure files.

Aspect	Detail / Explanation
Section A	VStack — Database & ORM (schema.ts, drizzle.config.ts)
Section B	VStack — Server Actions (GetChat.ts, GetAi.ts, GetUser.ts, payment.ts)
Section C	VStack — App Layout & Provider (layout.tsx, provider.tsx, globals.css)
Section D	VStack — Pages (page.tsx home, sign-in, tokens)
Section E	VStack — React Components (ChatView, EditorView, EditorSandpack, MainSidebar, NavbarAvatar)
Section F	Cloud IDE — Backend Server (server.ts inside Docker container)
Section G	Cloud IDE — Management API & Reverse Proxy (traefik/src/index.ts)
Section H	Cloud IDE — Infrastructure (Dockerfile, docker-compose.yml)
Section I	Cloud IDE — Next.js API Routes (/api/docker/start|stop|running)
Section J	Configuration Files (next.config.ts, drizzle.config.ts, package.json)

SECTION A — VSTACK: DATABASE SCHEMA

A.1 src/service/Database/schema.ts

The schema file is the single source of truth for the VStack database. It defines five tables using Drizzle ORM's pgTable helper with full TypeScript inference. The user table includes a tokens column for AI generation credits. The Chats table uses JSONB columns for flexible storage of message arrays and file tree objects. The session, account, and verification tables are required by Better Auth.

Aspect	Detail / Explanation
File	src/service/Database/schema.ts
Language	TypeScript
ORM	Drizzle ORM (drizzle-orm/pg-core)
Tables	user, Transaction, Chats, session, account, verification
Key design	JSONB for messages[] and files{}; relations() for type-safe joins; indexes on userId FKs
Auth tables	session, account, verification are managed by Better Auth; do not edit manually

src/service/Database/schema.ts [TypeScript]

```
1  import { relations } from "drizzle-orm";
2  import {
3    pgTable, text, timestamp, boolean,
4    integer, index, jsonb,
5  } from "drizzle-orm/pg-core";
6
7  // ——— USERS TABLE ————————————————————————————————————————
8  export const user = pgTable("user", {
9    id:           text("id").primaryKey(),
10   name:         text("name").notNull(),
11   email:        text("email").notNull().unique(),
12   emailVerified:boolean("email_verified").default(false).notNull(),
13   image:        text("image"),
14   createdAt:    timestamp("created_at").defaultNow().notNull(),
15   updatedAt:    timestamp("updated_at")
16                 .defaultNow()
17                 .$onUpdate(() => new Date())
18                 .notNull(),
```

```
19   userRole:     text("user_role").default("user"),
20   tokens:       integer("tokens").default(20),
21 });
22
23 // ——— TRANSACTIONS TABLE ——————————————————————————————————————
24 export const Transaction = pgTable("transction", {
25   id:           text("id").primaryKey(),
26   userid:       text("user_id").references(() => user.id),
27   tokenupdated: integer("tokenupdated").notNull(),
28   createdAt:    timestamp("created_at").defaultNow().notNull(),
29   orderid:      text("order_id").notNull(),
30   paymentid:    text("payment_id").notNull(),
31   payerid:      text("signature").notNull(),
32 });
33
34 // ——— CHATS TABLE ————————————————————————————————————————————————
35 export const Chats = pgTable("chats", {
36   id:       text("id").primaryKey(),
37   userid:   text("user_id").references(() => user.id),
38   chatid:   text("chat_id").notNull().unique(),
39   files:    jsonb("files"),
40   messages: jsonb("messages").array(),
41   template: text("template").notNull().default("react"),
42 });
43
44 // ——— SESSIONS TABLE (Better Auth) ———————————————————————————————
45 export const session = pgTable("session", {
46   id:           text("id").primaryKey(),
47   expiresAt:    timestamp("expires_at").notNull(),
48   token:        text("token").notNull().unique(),
49   createdAt:    timestamp("created_at").defaultNow().notNull(),
50   updatedAt:    timestamp("updated_at")
51                 .$onUpdate(() => new Date()).notNull(),
52   ipAddress:    text("ip_address"),
53   userAgent:    text("user_agent"),
54   userId:       text("user_id").notNull()
55                 .references(() => user.id, { onDelete: "cascade" }),
56 }, (table) => [index("session_userId_idx").on(table.userId)]);
57
58 // ——— ACCOUNTS TABLE (OAuth) ——————————————————————————————————————
59 export const account = pgTable("account", {
60   id:                   text("id").primaryKey(),
61   accountId:            text("account_id").notNull(),
62   providerId:           text("provider_id").notNull(),
63   userId:               text("user_id").notNull()
64                         .references(() => user.id, { onDelete: "cascade" }),
65   accessToken:          text("access_token"),
66   refreshToken:         text("refresh_token"),
```

```
67   idToken:              text("id_token"),
68   accessTokenExpiresAt: timestamp("access_token_expires_at"),
69   refreshTokenExpiresAt:timestamp("refresh_token_expires_at"),
70   scope:                text("scope"),
71   password:             text("password"),
72   createdAt:            timestamp("created_at").defaultNow().notNull(),
73   updatedAt:            timestamp("updated_at")
74                        .$onUpdate(() => new Date()).notNull(),
75 }, (table) => [index("account_userId_idx").on(table.userId)]);
76
77 // ——— RELATIONS ————————————————————————————————————————————————
78 export const userRelations = relations(user, ({ many }) => ({
79   sessions:     many(session),
80   accounts:     many(account),
81   transactions: many(Transaction),
82   chats:        many(Chats),
83 }));
84
85 export const chatsRelations = relations(Chats, ({ one }) => ({
86   user: one(user, {
87     fields:     [Chats.userid],
88     references: [user.id],
89   }),
90 }));
91
92 export const transactionRelations = relations(Transaction, ({ one }) => ({
93   user: one(user, {
94     fields:     [Transaction.userid],
95     references: [user.id],
96   }),
97 }));
```

SECTION B — VSTACK: SERVER ACTIONS

B.1 src/actions/GetChat.ts — Chat CRUD Operations

GetChat.ts contains four server actions for managing chat sessions. GetChat() finds or creates a chat record for a given chatId and user. UpdateChat() is the most critical function: it runs inside a database transaction to atomically update the messages/files columns AND decrement the user's token count by 1. getAllChats() returns the list of chat IDs for the sidebar. DeleteChat() removes a session by chatId.

Aspect	Detail / Explanation
File	src/actions/GetChat.ts
Language	TypeScript (Next.js Server Action — 'use server')
Key pattern	db.transaction() ensures UpdateChat + token decrement are atomic (either both succeed or both fail)

Auth check	Every action calls auth.api.getSession() first; returns 400 if unauthenticated
Token deduct	sql`${userTable.tokens} - 1` uses Drizzle's SQL template for safe arithmetic update

src/actions/GetChat.ts [TypeScript]

```
1  "use server";
2  import { and, eq, sql } from "drizzle-orm";
3  import { db } from "@/service/Database/index";
4  import { Chats, user as userTable } from "@/service/Database/schema";
5  import { FileStructure } from "@/lib/Types";
6  import { auth } from "@/service/Auth/auth";
7  import { headers } from "next/headers";
8
9  // ——— GET or CREATE a chat session ————————————————————————————
10 export async function GetChat({
11   chatid, UserMessage, template,
12 }: {
13   chatid: string; UserMessage: string; template: string;
14 }) {
15   const session = await auth.api.getSession({ headers: await headers() });
16   const user = session?.user;
17
18   try {
19     if (!user) return { status: 400, error: "User not authenticated" };
20
21     const existingChat = await db.query.Chats.findFirst({
22       where: eq(Chats.userid, user.id as string) &&
23              eq(Chats.chatid, chatid as string),
24     });
25
26     if (!existingChat) {
27       // Empty prompt → create blank chat record
28       if (UserMessage === "") {
29         await db.insert(Chats).values({
30           id: crypto.randomUUID(), chatid,
31           userid: user.id as string,
32           files: {}, messages: [], template,
33         });
34         return { status: 200, messages: [], files: {}, template };
35       }
36       // First message → insert with user message pre-loaded
37       await db.insert(Chats).values({
38         id: crypto.randomUUID(), chatid,
39         userid: user.id as string,
40         messages: [{ role: "user", content: UserMessage }],
41         files: {}, template,
42       });
43       return {
44         status: 200,
45         messages: [{ role: "user", content: UserMessage }],
```

```
46        files: {}, template,
47      };
48    }
49    // Existing chat → return persisted state
50    return {
51      status: 200,
52      messages: existingChat.messages,
53      files:    existingChat.files,
54      template: existingChat.template,
55    };
56  } catch (error) {
57    console.log(error);
58    return { error: "User not authenticated", status: 400 };
59  }
60 }
61
62 // ——— UPDATE chat messages + files, DEDUCT one token (transaction) ─
63 export async function UpdateChat(
64   chatid: string, message: [], file: FileStructure,
65 ) {
66   const session = await auth.api.getSession({ headers: await headers() });
67   const user = session?.user;
68   if (!user) return { status: 400, error: "User not authenticated" };
69
70   try {
71     await db.transaction(async (tx) => {
72       // 1. Update chat messages and generated files
73       await tx.update(Chats)
74         .set({ messages: message, files: file })
75         .where(
76           and(eq(Chats.userid, user.id as string), eq(Chats.chatid, chatid))
77         );
78       // 2. Atomically decrement user token count
79       await tx.update(userTable)
80         .set({ tokens: sql`${userTable.tokens} - 1` })
81         .where(eq(userTable.id, user.id as string));
82     });
83   } catch (error) {
84     console.log(error);
85     return { status: 400, error: "Internal server error" };
86   }
87 }
88
89 // ——— GET all chat IDs for the current user ——————————————————————
90 export async function getAllChats() {
91   const session = await auth.api.getSession({ headers: await headers() });
92   const user = session?.user;
93   if (!user) return 400;
94   try {
95     const data = await db
96       .select({ chatid: Chats.chatid })
```

```
97        .from(Chats)
98        .where(eq(Chats.userid, user.id as string));
99      return data;
100    } catch (error) { console.log(error); return 400; }
101  }
102
103  // ——— DELETE a chat by chatid ——————————————————————————————————
104  export async function DeleteChat(chatid: string) {
105    const session = await auth.api.getSession({ headers: await headers() });
106    const user = session?.user;
107    if (!user) return 400;
108    try {
109      await db.delete(Chats).where(
110        and(eq(Chats.userid, user.id as string), eq(Chats.chatid, chatid))
111      );
112      return 200;
113    } catch (error) { console.log(error); return 400; }
114  }
```

B.2 src/actions/GetAi.ts — AI Generation Pipeline

ProcessChat() is the heart of VStack. It orchestrates the full AI generation cycle in strict order: authenticate → check token balance → call Gemini chat session → call Gemini code session → parse JSON → persist → return result. Two separate Gemini sessions are used to keep conversational text and structured code output cleanly separated.

Aspect	Detail / Explanation
File	src/actions/GetAi.ts
Language	TypeScript (Next.js Server Action)
AI provider	Google Gemini AI via @google/generative-ai SDK
Dual sessions	chatSession produces readable text; codeSession produces JSON-only file tree
JSON parsing	JSON.parse(codeText) — if AI returns malformed JSON this throws; caught by outer try/catch
Token guard	GetTokens() called before any AI call; returns 403 if balance is zero
Persistence	UpdateChat() called after successful parse; atomically saves output + decrements token

📄 **src/actions/GetAi.ts** [TypeScript]

```
1  "use server";
2  import { chatSession, codeSession } from "@/service/Ai";
3  import { GetTokens } from "./GetUser";
4  import { UpdateChat } from "./GetChat";
5  import { auth } from "@/service/Auth/auth";
6  import { headers } from "next/headers";
7  import { React_Chat_Prompt, React_Code_Prompt } from "@/lib/Constant";
8  import { FileStructure, Message } from "@/lib/Types";
9
```

```
10  // ——— Single AI chat turn (returns conversational text) ——————————
11  export async function GetAiMessage(Message: string) {
12    try {
13      const data = await chatSession.sendMessage(Message);
14      return { status: 200, content: data.response.text() };
15    } catch (error) {
16      console.log(error);
17      return { status: 400, content: "Failed to generate AI code" };
18    }
19  }
20
21  // ——— Full generation pipeline ————————————————————————————————————
22  // 1. Auth check  2. Token check  3. Gemini chat  4. Gemini code
23  // 5. Parse JSON  6. Persist to DB  7. Return to client
24  export async function ProcessChat({
25    chatid, messages,
26  }: {
27    chatid: string; messages: Message[];
28  }) {
29    const session = await auth.api.getSession({ headers: await headers() });
30    const user = session?.user;
31    if (!user) return { status: 400, error: "User not authenticated" };
32
33    try {
34      // ── Step 1: Token guard ——————————————————————————————————————
35      const tokenStatus = await GetTokens();
36      if (tokenStatus.status !== 200 || (tokenStatus.tokens ?? 0) <= 0) {
37        return { status: 403, error: "No tokens left" };
38      }
39
40      const userChat = messages[messages.length - 1].content;
41
42      // ── Step 2: Gemini chat session → conversational explanation ──
43      const aiResponse = await GetAiMessage(userChat + React_Chat_Prompt);
44      if (aiResponse.status !== 200) {
45        return { status: 400, error: "Failed to generate AI response" };
46      }
47      const aiMessage: Message = { role: "assistant", content: aiResponse.content };
48      const updatedMessages = [...messages, aiMessage];
49
50      // ── Step 3: Gemini code session → structured JSON file tree ———
51      const codePrompt  = userChat + "" + React_Code_Prompt;
52      const codeResult  = await codeSession.sendMessage(codePrompt);
53      const codeText    = codeResult.response.text();
54
55      // ── Step 4: Parse JSON output from AI ————————————————————————
56      const parsedData  = JSON.parse(codeText);        // throws if malformed
57      const newFiles    = parsedData.files as FileStructure;
58
59      // ── Step 5: Persist messages + files, decrement token ————————
60      await UpdateChat(chatid, updatedMessages as [], newFiles);
```

```
61
62      return { status: 200, messages: updatedMessages, files: newFiles };
63    } catch (error) {
64      console.error("Error in ProcessChat:", error);
65      return { status: 500, error: "Internal server error" };
66    }
67  }
```

B.3 src/actions/GetUser.ts — Token Balance Query

GetTokens() is a lightweight server action that queries only the tokens column from the users table for the currently authenticated user. It is called at the start of every AI generation to gate access and before rendering the tokens page to display the current balance.

Aspect	Detail / Explanation
File	src/actions/GetUser.ts
Purpose	Read-only token balance check
Returns	{ tokens: number, status: 200 } or { status: 400 }
Used by	ProcessChat (gate check) and Tokens page (display)

src/actions/GetUser.ts [TypeScript]

```
1  "use server";
2  import { eq } from "drizzle-orm";
3  import { db } from "@/service/Database/index";
4  import { user as userTable } from "@/service/Database/schema";
5  import { auth } from "@/service/Auth/auth";
6  import { headers } from "next/headers";
7
8  // ——— Fetch the current authenticated user's token balance ——————————
9  export async function GetTokens() {
10   const session = await auth.api.getSession({ headers: await headers() });
11   const user = session?.user;
12   try {
13     if (!user) return { status: 400 };
14
15     const existingTokens = await db
16       .select({ tokens: userTable.tokens })
17       .from(userTable)
18       .where(eq(userTable.id, user.id as string));
19
20     if (existingTokens.length === 0) return { tokens: 0, status: 200 };
21     return { tokens: existingTokens[0].tokens, status: 200 };
22   } catch (error) {
23     return { error, status: 401 };
24   }
25 }
```

B.4 src/actions/payment.ts — PayPal Token Credit

updateTokens() is called after a successful PayPal payment. It runs a database transaction that inserts a full audit record into the Transaction table (order ID, payment ID, payer ID, token amount) and then increments the user's tokens column by the purchased amount. Using sql`tokens + ${tokens}` ensures the increment is race-condition safe.

Aspect	Detail / Explanation
File	src/actions/payment.ts
Language	TypeScript (Next.js Server Action)
Key pattern	db.transaction() wraps both INSERT (audit) and UPDATE (token credit)
Audit trail	Every purchase stores orderId, paymentId, payerId for compliance
Increment	sql`${userTable.tokens} + ${tokens}` — atomic SQL arithmetic, not read-modify-write

src/actions/payment.ts [TypeScript]

```
1  "use server";
2  import { eq, sql } from "drizzle-orm";
3  import { db } from "@/service/Database/index";
4  import { Transaction, user as userTable } from "@/service/Database/schema";
5  import { auth } from "@/service/Auth/auth";
6  import { headers } from "next/headers";
7
8  // ——— Credit tokens after successful PayPal payment ————————————
9  // Runs inside a DB transaction:
10 //   1. Insert transaction record (audit trail)
11 //   2. Increment user.tokens by purchased amount
12 export async function updateTokens({
13   tokens, orderId, paymentId, payerId,
14 }: {
15   tokens: number; orderId: string;
16   paymentId: string; payerId: string;
17 }) {
18   const session = await auth.api.getSession({ headers: await headers() });
19   const user = session?.user;
20
21   try {
22     if (!user) return { status: 400, error: "User not authenticated" };
23
24     await db.transaction(async (tx) => {
25       // 1. Audit log: persist PayPal transaction details
26       await tx.insert(Transaction).values({
27         id:          crypto.randomUUID(),
28         orderid:     orderId,
29         paymentid:   paymentId,
30         payerid:     payerId,
31         tokenupdated: tokens,
32         userid:      user.id,
33       });
```

```
34      // 2. Credit the purchased tokens to user account
35      await tx.update(userTable)
36        .set({ tokens: sql`${userTable.tokens} + ${tokens}` })
37        .where(eq(userTable.id, user.id as string));
38    });
39
40    return { status: 200, message: "success" };
41  } catch (error) {
42    console.log(error);
43    return { error: "Transaction failed", status: 400 };
44  }
45 }
```

SECTION C — VSTACK: APP LAYOUT & GLOBAL STYLES

C.1 src/components/provider.tsx — Root Context Provider

Provider.tsx is the outermost client component wrapper. It provides four React contexts to the entire application, wraps everything in the PayPal SDK provider (required at top level), renders the persistent NavBar and Sidebar, conditionally renders the AccountBilling modal, and shows a mobile-not-supported message. The dark gradient background is applied here as a global layout class.

Aspect	Detail / Explanation
File	src/components/provider.tsx
Pattern	Context composition — four providers nested in one component
PayPal	PayPalScriptProvider must sit above all PayPalButtons components
Mobile	App is hidden on mobile (md:hidden) — IDE-style layout requires desktop viewport
Routing	usePathname() used to conditionally show/hide MainSidebar and BottomBar

src/components/provider.tsx [TypeScript (React)]

```
1  "use client";
2  import React from "react";
3  import MainSidebar from "@/components/MainSidebar";
4  import MainNavBar from "@/components/MainNavBar";
5  import BottomBar from "@/components/BottomBar";
6  import AccountBilling from "@/components/account-billing";
7  import { Toaster } from "react-hot-toast";
8  import { PayPalScriptProvider } from "@paypal/react-paypal-js";
9  import {
10   AccountBillingContext, SandBoxContext,
11   TemplateContext, UserMessageContext,
12 } from "@/lib/Context";
13 import { usePathname } from "next/navigation";
14
15 // ——— Root Provider: wraps the entire app with all contexts ————————
```

```
// Contexts provided:
//   TemplateContext  – active stack template (react, vue, nextjs…)
//   UserMessageContext – the prompt typed on the home page
//   SandBoxContext   – sandbox action state (deploy / export)
//   AccountBillingContext – modal visibility for account / billing
export default function Provider({ children }: { children: React.ReactNode }) {
  const [sandBox, setsandBox] = React.useState({ sandBoxType: "", timeStamp: 0 });
  const [UserMessage, SetUserMessage] = React.useState("");
  const [accountBilling, setaccountBilling] = React.useState({
    accountBillingType: "", is: false,
  });
  const [template, setTemplate] = React.useState<string>("react");
  const pathname = usePathname();

  return (
    // PayPal SDK wrapper – must sit at top level
    <PayPalScriptProvider options={{ clientId: process.env.NEXT_PUBLIC_PAYPAL_KEY_ID! }}>
      <TemplateContext.Provider value={{ template, setTemplate }}>
        <UserMessageContext.Provider value={{ UserMessage, SetUserMessage }}>
          <SandBoxContext.Provider value={{ sandBox, setsandBox }}>
            <AccountBillingContext.Provider value={{ accountBilling, setaccountBilling }}>

              {/* Mobile fallback – IDE not supported on mobile */}
              <div className="relative flex min-h-screen flex-col items-center
                        justify-center bg-gradient-to-br
                        from-[#141414] via-[#222222] to-[#383737] md:hidden">
                <h1>Not available for mobile devices</h1>
              </div>

              {/* Desktop layout */}
              <div className="relative hidden min-h-screen flex-col
                        bg-gradient-to-br from-[#141414] via-[#222222]
                        to-[#383737] md:flex">
                <Toaster position="top-center" reverseOrder={false} />
                {accountBilling.is && <AccountBilling />}
                <MainNavBar />
                {!pathname.startsWith("/tokens") && <MainSidebar />}
                {children}
                {!pathname.startsWith("/chat") && <BottomBar />}
              </div>

            </AccountBillingContext.Provider>
          </SandBoxContext.Provider>
        </UserMessageContext.Provider>
      </TemplateContext.Provider>
    </PayPalScriptProvider>
  );
}
```

C.2 src/app/globals.css — Global Styles & Design Tokens

The globals.css file defines the CSS custom property design token system used by shadcn/ui components throughout VStack. All colour values are in HSL format without the hsl() wrapper, which is the format expected by shadcn/ui's Tailwind plugin. A dark theme is enforced globally — there is no light mode toggle. The Syne Mono font is imported for brand typography in the sign-in screen.

src/app/globals.css [CSS]

```
/* src/app/globals.css */
/* Google Font import for Syne Mono (used in brand typography) */
@import url("https://fonts.googleapis.com/css2?family=Syne+Mono&display=swap");

@tailwind base;
@tailwind components;
@tailwind utilities;

/* ── CSS Custom Properties (shadcn/ui design token system) ──────────── */
@layer base {
  :root {
    /* Dark theme color tokens (used by shadcn/ui components) */
    --background:           0 0% 3.9%;    /* near-black body bg */
    --foreground:           0 0% 98%;     /* near-white text */
    --card:                 0 0% 3.9%;
    --card-foreground:      0 0% 98%;
    --popover:              0 0% 3.9%;
    --popover-foreground:   0 0% 98%;
    --primary:              0 0% 98%;
    --primary-foreground:   0 0% 9%;
    --secondary:            0 0% 14.9%;   /* dark gray surfaces */
    --secondary-foreground: 0 0% 98%;
    --muted:                0 0% 14.9%;
    --muted-foreground:     0 0% 63.9%;
    --accent:               0 0% 14.9%;
    --accent-foreground:    0 0% 98%;
    --destructive:          0 62.8% 30.6%;
    --destructive-foreground:0 0% 98%;
    --border:               0 0% 14.9%;
    --input:                0 0% 14.9%;
    --ring:                 0 0% 83.1%;
    --chart-1:              220 70% 50%;
    --chart-2:              160 60% 45%;
    --chart-3:              30 80% 55%;
    --chart-4:              280 65% 60%;
    --chart-5:              340 75% 55%;
    /* Sidebar tokens */
    --sidebar-background:   0 0% 98%;
    --sidebar-foreground:   240 5.3% 26.1%;
    --sidebar-primary:      240 5.9% 10%;
    --sidebar-primary-fg:   0 0% 98%;
```

```
42      --sidebar-border:       220 13% 91%;
43      --sidebar-ring:         217.2 91.2% 59.8%;
44    }
45    .dark {
46      --sidebar-background:   240 5.9% 10%;
47      --sidebar-foreground:   240 4.8% 95.9%;
48      --sidebar-primary:      224.3 76.3% 48%;
49      --sidebar-border:       240 3.7% 15.9%;
50      --sidebar-ring:         217.2 91.2% 59.8%;
51    }
52  }
53
54  @layer base {
55    * { @apply border-border; }
56    body { @apply bg-background text-foreground; }
57  }
```

SECTION D — VSTACK: APPLICATION PAGES

D.1 src/app/page.tsx — Home Page

The home page is the primary entry point. It renders the animated headline (Cover component), the prompt textarea, the forward submit button (conditionally visible when text is non-empty), the suggestion chip grid, and the Stacks framework icon row. All user interactions that require authentication redirect to /sign-in if the user session is absent.

Aspect	Detail / Explanation
File	src/app/page.tsx
Type	Next.js Client Component ('use client')
State	text — current textarea value (local useState)
Context	UserMessageContext.SetUserMessage — stores prompt for /chat/[id] to consume
Routing	Math.random().toString(36).slice(2) generates a unique 8-char chat ID per session

📄 **src/app/page.tsx** [TypeScript (React)]

```
1  "use client";
2  import { useSession } from "@/lib/auth-client";
3  import React from "react";
4  import { UserMessageContext } from "@/lib/Context";
```

```
import { useRouter } from "next/navigation";
import { Textarea } from "@/components/ui/textarea";
import { Forward } from "lucide-react";
import { cn } from "@/lib/utils";
import { Suggestions } from "@/lib/Constant";
import { Cover } from "@/components/ui/cover";
import Stacks from "@/components/stacks";
import toast from "react-hot-toast";

export default function Home() {
  const { SetUserMessage } = React.useContext(UserMessageContext);
  const router = useRouter();
  const session = useSession();
  const [text, setText] = React.useState<string>("");
  const user = session.data?.user;

  // On custom prompt submit → store in context, navigate to new chat
  const handleSubmit = async () => {
    if (!user) { router.push("/sign-in"); return; }
    const chatid = Math.random().toString(36).slice(2);
    SetUserMessage(text);
    router.push(`/chat/${chatid}`);
  };

  // On suggestion chip click → same flow with pre-filled prompt
  const handleGenerate = async (suggestion: string) => {
    if (!user) { router.push("/sign-in"); return; }
    const chatid = Math.random().toString(36).slice(2);
    SetUserMessage(suggestion);
    router.push(`/chat/${chatid}`);
  };

  // Stack template selection (coming soon)
  const handleStacks = async (template: string) => {
    if (!user) { router.push("/sign-in"); return; }
    toast("coming soon");
  };

  return (
    <div className="flex h-full w-full flex-grow flex-col items-center justify-center">
      {/* Headline with animated cover effect */}
      <h1 className="align-middle text-3xl">
        Generate your <Cover className="cursor-none">Own Website</Cover>
      </h1>
      <p className="my-1 mb-4 font-bold opacity-50">
        Generate, preview, edit code and deploy.
      </p>

      {/* Main prompt textarea */}
      <div className="flex h-[200px] w-[450px] flex-col rounded-xl
                  border border-[#3a3a3a] bg-transparent
```

```
56                  shadow-xl backdrop-blur-3xl">
57          <Textarea
58            inputMode="text"
59            onChange={(e) => setText(e.target.value)}
60            placeholder="See the magic ....."
61            className="h-full w-full resize-none border-none
62                       font-bold focus-visible:ring-0"
63          />
64          <div className="flex h-[50px] w-full flex-grow items-center justify-between">
65            <div className="flex flex-grow"></div>
66            <div className="mr-3 flex items-center justify-center">
67              {/* Forward button appears only when text is present */}
68              <div
69                onClick={handleSubmit}
70                className={cn(text.length === 0 && "hidden", "text-white")}
71              >
72                <Forward className="h-7 w-7 cursor-pointer" />
73              </div>
74            </div>
75          </div>
76        </div>
77
78        {/* Suggestion chips */}
79        <div className="mt-5 flex w-[600px] flex-wrap items-center justify-center gap-2">
80          {Suggestions.map((suggestion, index) => (
81            <p
82              onClick={() => handleGenerate(suggestion)}
83              key={index}
84              className="group flex cursor-pointer gap-3 truncate rounded-xl
85                         border border-[#3a3a3a] p-1 px-2 text-[12px]"
86            >
87              {suggestion}
88            </p>
89          ))}
90        </div>
91
92        {/* Framework stack icons */}
93        <Stacks handleStacks={handleStacks} />
94      </div>
95    );
96  }
```

D.2 src/app/sign-in/page.tsx — OAuth Sign-In Page

The sign-in page is intentionally minimal: it supports only OAuth providers (GitHub and Google). Email/password sign-in was omitted to reduce friction and avoid password management complexity. Better Auth's signIn.social() initiates the provider-specific OAuth flow and handles the full callback and session creation cycle automatically.

Aspect	Detail / Explanation
File	src/app/sign-in/page.tsx
Type	Next.js Client Component
Auth	Better Auth signIn.social({ provider, callbackURL }) — no manual token handling needed
Providers	github, google — configured in src/service/Auth/auth.ts
Redirect	callbackURL: '/' — returns to home page after successful OAuth

src/app/sign-in/page.tsx [TypeScript (React)]

```
1  "use client";
2  import React from "react";
3  import { signIn } from "@/lib/auth-client";
4  import { Button } from "@/components/ui/button";
5  import { Github } from "lucide-react";
6
7  // ——— OAuth-only Sign-In page ——————————————————————————————
8  // Supports GitHub and Google providers via Better Auth social login.
9  // On success, Better Auth redirects to callbackURL ("/").
10 export default function SignInPage() {
11   const handleSignIn = async (provider: "github" | "google") => {
12     await signIn.social({
13       provider,
14       callbackURL: "/",  // redirect back to home after OAuth
15     });
16   };
17
18   return (
19     <div className="flex flex-col items-center justify-center h-full w-full flex-grow">
20       <div className="flex w-[400px] flex-col rounded-xl
21                       border border-[#3a3a3a] bg-[#1a1a1a]/50
22                       p-8 shadow-2xl backdrop-blur-3xl">
23         <h1 className="mb-6 text-center text-3xl font-bold text-white">
24           Sign in to <span className="font-syne">Vstack</span>
25         </h1>
26         <p className="mb-8 text-center text-sm text-gray-400">
27           Choose your preferred provider to continue building.
28         </p>
29         <div className="flex flex-col gap-4">
30
31           {/* GitHub OAuth Button */}
32           <Button
33             variant="outline"
34             className="flex items-center justify-center gap-3
35                        border-[#3a3a3a] bg-transparent py-6 text-white
36                        hover:bg-[#3a3a3a]"
37             onClick={() => handleSignIn("github")}
```

```
38          >
39            <Github className="h-5 w-5" />
40            Continue with GitHub
41          </Button>
42
43          {/* Google OAuth Button */}
44          <Button
45            variant="outline"
46            className="flex items-center justify-center gap-3
47                       border-[#3a3a3a] bg-transparent py-6 text-white
48                       hover:bg-[#3a3a3a]"
49            onClick={() => handleSignIn("google")}
50          >
51            {/* Inline Google SVG icon */}
52            <svg className="h-5 w-5" viewBox="0 0 488 512"
53                 xmlns="http://www.w3.org/2000/svg">
54              <path fill="currentColor"
55                d="M488 261.8C488 403.3 391.1 504 248 504
56                   110.8 504 0 393.2 0 256S110.8 8 248 8
57                   c66.8 0 123 24.5 166.3 64.9l-67.5 64.9
58                   C258.5 52.6 94.3 116.6 94.3 256
59                   c0 86.5 69.1 156.6 153.7 156.6
60                   98.2 0 135-70.4 140.8-106.9H248v-85.3
61                   h236.1c2.3 12.7 3.9 24.9 3.9 41.4z"/>
62            </svg>
63            Continue with Google
64          </Button>
65        </div>
66        <p className="mt-8 text-center text-xs text-gray-500">
67          By signing in, you agree to our Terms of Service and Privacy Policy.
68        </p>
69      </div>
70    </div>
71  );
72  }
```

D.3 src/app/tokens/page.tsx — Token Purchase Page

The tokens page renders PayPal payment buttons for each token package defined in the allBuy constant. PayPal's createOrder() creates the order on PayPal's servers; onApprove() fires after the user completes payment and triggers the updateTokens() server action. The PayPal SDK renders its own button UI — it is not a custom button, which ensures PCI compliance.

Aspect	Detail / Explanation
File	src/app/tokens/page.tsx
Type	Next.js Client Component

SDK	@paypal/react-paypal-js — PayPalButtons renders PayPal's native UI
Flow	createOrder → PayPal UI → user pays → onApprove → updateTokens() server action
Disabled	PayPalButtons disabled={!user} — prevents payment if not logged in
Loading	Session.isPending shows spinner while auth state resolves

src/app/tokens/page.tsx [TypeScript (React)]

```tsx
"use client";
import { updateTokens } from "@/actions/payment";
import { allBuy } from "@/lib/Constant";
import { PayPalButtons } from "@paypal/react-paypal-js";
import { Loader } from "lucide-react";
import { authClient } from "@/lib/auth-client";
import toast from "react-hot-toast";
import { useRouter } from "next/navigation";

// ——— Token Purchase page using PayPal ————————————————————————
// allBuy = array of { price, tokens, description } packages
// PayPalButtons: SDK renders the PayPal UI natively
// On approval: calls updateTokens() server action to credit tokens
export default function Pricing() {
  const notify = () => toast("payment failed");
  const Session = authClient.useSession();
  const router = useRouter();
  const user = Session.data?.user;

  const onPaymentSuccess = async ({
    orderId, paymentId, payerId, tokens,
  }: {
    orderId: string; paymentId: string;
    payerId: string; tokens: number;
  }) => {
    if (!user) return router.push("/sign-in");
    await updateTokens({ tokens, orderId, paymentId, payerId });
  };

  return Session.isPending ? (
    <div className="flex min-h-[calc(100vh-95px)] w-full
                    items-center justify-center">
      <Loader className="animate-spin" />
    </div>
  ) : (
    <div className="flex min-h-[calc(100vh-95px)] w-full
                    items-center justify-center">
      <div className="flex h-full w-full flex-col">
        <div className="flex w-full items-center justify-center px-6 py-4">
          <span className="mb-1 mt-1 text-2xl font-semibold">Buy tokens</span>
        </div>
```

```
42
43        {/* Token package cards */}
44        <div className="flex flex-col justify-center gap-4 px-3 py-3 lg:flex-row">
45          {allBuy.map((item, index) => (
46            <div key={index}
47                 className="flex flex-1 flex-col justify-center gap-5
48                            rounded-xl border border-[#353434] px-6 pb-10 pt-6
49                            text-sm shadow-xl">
50              <div className="flex flex-col gap-1">
51                <p className="text-xl font-semibold">Buy Tokens</p>
52                <div className="flex items-baseline gap-1.5">
53                  <p className="text-5xl">{item.price}</p>
54                  <p className="text-xs">USD</p>
55                </div>
56                <p className="mt-2 min-h-[100px] text-base">
57                  {item.description}
58                </p>
59              </div>
60
61              {/* PayPal native button – disabled if not signed in */}
62              <PayPalButtons
63                disabled={!user}
64                onCancel={notify}
65                onApprove={async (data) => {
66                  await onPaymentSuccess({
67                    orderId:   data.orderID,
68                    paymentId: data.paymentID!,
69                    payerId:   data.payerID!,
70                    tokens:    item.tokens,
71                  });
72                }}
73                createOrder={(data, actions) =>
74                  actions.order.create({
75                    purchase_units: [{
76                      description: item.description,
77                      // @ts-ignore
78                      amount: { value: item.price, currency_code: "USD" },
79                    }],
80                  })
81                }
82                style={{ shape: "pill", color: "black", layout: "vertical" }}
83              />
84            </div>
85          ))}
86        </div>
87      </div>
88    </div>
89  );
90 }
```

SECTION E — VSTACK: REACT COMPONENTS

E.1 src/components/ChatView.tsx — Chat Conversation Panel

ChatView renders the left panel of the generation screen. It displays the alternating user/AI message list with auto-scroll to the latest message on every update. The most recently received AI message renders with a TypingAnimation effect for a conversational feel. A fixed textarea at the bottom of the panel accepts follow-up prompts.

Aspect	Detail / Explanation
File	src/components/ChatView.tsx
Auto-scroll	useRef + useEffect on Message array; scrollTop = scrollHeight
Markdown	react-markdown renders AI responses with proper headings, lists, code blocks
Animation	TypingAnimation applied to last message only when animation prop is true
Input	Textarea disabled during codeLoading; Forward icon hidden when empty

src/components/ChatView.tsx [TypeScript (React)]

```
1  import { useRef, useEffect } from "react";
2  import { Forward } from "lucide-react";
3  import { Textarea } from "./ui/textarea";
4  import { cn } from "@/lib/utils";
5  import { Message } from "@/lib/Types";
6  import ReactMarkdown from "react-markdown";
7  import { Avatar, AvatarFallback, AvatarImage } from "./ui/avatar";
8  import { useSession } from "@/lib/auth-client";
9  import TypingAnimation from "@/components/ui/typing-animation";
10
11 // ——— Chat conversation panel ————————————————————————————
12 // Props:
13 //   codeLoading  – disables input while AI is generating
14 //   animation    – triggers typing animation on latest AI message
15 //   Message      – array of {role, content} message objects
16 //   HandleUpdate – callback to submit follow-up prompt
17 export default function ChatView({
18   codeLoading, animation, Message, HandleUpdate, text, setText,
19 }: {
20   codeLoading: boolean; animation: boolean; Message: Message[];
21   HandleUpdate: () => void;
22   text: string; setText: React.Dispatch<React.SetStateAction<string>>;
23 }) {
24   const { data: session } = useSession();
25   const user = session?.user;
26   const chatContainerRef = useRef<HTMLDivElement>(null);
27
28   // Auto-scroll to bottom on new message
29   useEffect(() => {
30     if (chatContainerRef.current) {
31       chatContainerRef.current.scrollTop =
```

```
32        chatContainerRef.current.scrollHeight;
33      }
34    }, [Message]);
35
36    return (
37      <div className="flex h-[calc(100vh-100px)] w-[450px] flex-grow
38                    flex-col items-center overflow-hidden rounded-xl">
39        {/* Scrollable message list */}
40        <div ref={chatContainerRef}
41          className="mb-[200px] flex w-full flex-col items-center
42                      overflow-y-scroll scrollbar-hide">
43          {Message.map((item, index) => {
44            const isLastMessage = index === Message.length - 1;
45            return (
46              <div
47                className={cn(
48                  item.role === "user" ? "font-bold" : "text-base",
49                  "my-2 flex w-[400px] flex-shrink rounded-xl border",
50                )}
51                key={index}
52              >
53                {item.role === "user" ? (
54                  // User bubble: avatar + markdown content
55                  <>
56                    <Avatar className="m-2 border border-[#3a3a3a]">
57                      <AvatarImage src={user?.image || "/default-avatar.png"} />
58                      <AvatarFallback>CN</AvatarFallback>
59                    </Avatar>
60                    <div className="p-3">
61                      <ReactMarkdown>{item.content}</ReactMarkdown>
62                    </div>
63                  </>
64                ) : item.role === "assistant" && animation && isLastMessage ? (
65                  // Latest AI message: typing animation
66                  <TypingAnimation className="p-3 text-base" duration={10}>
67                    {item.content}
68                  </TypingAnimation>
69                ) : (
70                  // Previous AI messages: static markdown
71                  <div className="p-3 text-base">
72                    <ReactMarkdown>{item.content}</ReactMarkdown>
73                  </div>
74                )}
75              </div>
76            );
77          })}
78        </div>
79
80        {/* Fixed follow-up input at bottom of panel */}
81        <div className="absolute bottom-1 left-0 z-10 mb-2 ml-[10px]
82                      flex h-[200px] w-[450px] flex-col rounded-xl
```

```
83                  border border-[#3a3a3a] bg-transparent backdrop-blur-3xl">
84          <Textarea
85            disabled={codeLoading}
86            value={text}
87            onChange={(e) => setText(e.target.value)}
88            placeholder="See the magic ........"
89            className="h-full w-full resize-none border-none
90                    font-bold focus-visible:ring-0"
91          />
92          <div className="flex h-[50px] w-full flex-grow items-center justify-end">
93            <div className="mr-3">
94              <div
95                onClick={HandleUpdate}
96                className={cn(text.length === 0 && "hidden", "text-white")}
97              >
98                <Forward className="h-7 w-7 cursor-pointer" />
99              </div>
100           </div>
101         </div>
102       </div>
103     </div>
104   );
105 }
```

SECTION F — CLOUD IDE: BACKEND SERVER (IN-CONTAINER)

F.1 packages/server/src/server.ts — WebSocket + REST Server

server.ts runs inside each user's Docker container on port 9000. It is the core of the Cloud IDE backend. It spawns a single node-pty bash process and broadcasts its output to all Socket.IO clients. It exposes REST endpoints for file tree reading, file content reading/writing, and terminal command writing. chokidar watches the user directory and emits real-time file change events.

Aspect	Detail / Explanation
File	packages/server/src/server.ts
Runtime	Node.js 22 inside Docker container (node:22-alpine image)
Terminal I/O	node-pty spawns bash; onData → io.emit('terminal:data'); POST /api/terminal → pty.write()
File tree	getFileListTree(): async recursive readdir; dirs → object, files → null
File watch	chokidar.watch(USER_DIR, {ignored: /^\./, persistent: true}) → io.emit('file-change')

CORS	origin:'*' required — Traefik proxies from a different subdomain
Port	9000 — mapped by Traefik to userid.localhost

packages/server/src/server.ts [TypeScript (Node.js)]

```
1  // packages/server/src/server.ts
2  // Runs INSIDE the Docker container on port 9000.
3  // Provides: REST file API + WebSocket terminal via Socket.IO + node-pty
4  import http from "http";
5  import chokidar from "chokidar";
6  import express from "express";
7  import { Server as SocketServer } from "socket.io";
8  import fs from "fs/promises";
9  import fsSync from "fs";
10 import path from "path";
11 import pty from "node-pty";
12 import cors from "cors";
13 import { fileURLToPath } from "url";
14 import { dirname } from "path";
15
16 const __filename = fileURLToPath(import.meta.url);
17 const __dirname  = dirname(__filename);
18
19 // Working directory: all user files live here
20 const USER_DIR = path.join(process.cwd(), "user");
21 if (!fsSync.existsSync(USER_DIR)) {
22   fsSync.mkdirSync(USER_DIR, { recursive: true });
23 }
24
25 const app    = express();
26 const server = http.createServer(app);
27
28 // Socket.IO with CORS open (for cross-subdomain Traefik routing)
29 const io = new SocketServer(server, { cors: { origin: "*", methods: ["GET","POST"] } });
30 app.use(cors());
31 app.use(express.json());
32
33 // ─── Spawn ONE pseudo-terminal (bash) for this container ──────────
34 const ptyProcess = pty.spawn("bash", [], {
35   name: "xterm-color",
36   cwd:  USER_DIR,
37   env:  process.env,
38 });
39
```

```
// Broadcast terminal output to ALL connected clients
ptyProcess.onData((data) => { io.emit("terminal:data", data); });

io.on("connection", (socket) => {
  console.log(`Socket connected: ${socket.id}`);
  socket.on("disconnect", () => console.log(`Socket disconnected: ${socket.id}`));
});

// ─── REST: write keystroke input to the pty process ──────────────────
app.post("/api/terminal", (req, res) => {
  const { data } = req.body;
  ptyProcess.write(data);
  res.sendStatus(200);
});

// ─── REST: health check (used by frontend to verify liveness) ────
app.get("/health", (req, res) => { res.sendStatus(200); });

// ─── REST: recursive file tree ──────────────────────────────────────────
app.get("/files", async (req, res): Promise<any> => {
  try {
    const tree = await getFileListTree(USER_DIR);
    return res.json(tree);
  } catch (error) {
    console.error("Error fetching file tree:", error);
    res.status(500).send("Error fetching file tree");
  }
});

// ─── REST: read a single file ───────────────────────────────────────────
app.get("/files/:filePath", async (req, res) => {
  try {
    const { filePath } = req.params;
    const fileContent = await fs.readFile(
      path.join(USER_DIR, filePath), "utf-8"
    );
    res.json(fileContent);
  } catch (error) {
    res.status(500).send("Error fetching file content");
  }
});

// ─── REST: save a single file ───────────────────────────────────────────
app.post("/files/:filePath", async (req, res) => {
  try {
```

```
85      const { filePath } = req.params;
86      const { content }  = req.body;
87      await fs.writeFile(path.join(USER_DIR, filePath), content, "utf-8");
88      res.status(200).send("File saved successfully");
89    } catch (error) {
90      res.status(500).send("Error saving file content");
91    }
92  });
93
94  // ——— Build nested directory tree (dirs = objects, files = null) ———
95  async function getFileListTree(dir: any) {
96    const tree = {};
97    async function treelist(curdir: any, currtree: any) {
98      const files = await fs.readdir(curdir);
99      await Promise.all(files.map(async (file) => {
100       const filepath = path.join(curdir, file);
101       const stat     = await fs.stat(filepath);
102       if (stat.isDirectory()) {
103         currtree[file] = {};
104         await treelist(filepath, currtree[file]);
105       } else {
106         currtree[file] = null;   // leaf node = file
107       }
108     }));
109   }
110   await treelist(dir, tree);
111   return tree;
112 }
113
114 // ——— Watch USER_DIR and notify all clients on any change ——————————
115 const watcher = chokidar.watch(USER_DIR, {
116   ignored:    /(^|[/\])../,   // ignore hidden files
117   persistent: true,
118 });
119 watcher.on("all", (event, path) => {
120   console.log(`File ${event}: ${path}`);
121   io.emit("file-change", { event, path });
122 });
123
124 server.listen(9000, () => { console.log("Server started on port 9000"); });
```

SECTION G — CLOUD IDE: MANAGEMENT API & TRAEFIK

G.1 packages/traefik/src/index.ts — Docker Container Manager

index.ts is the Management API that runs on the HOST machine (not inside containers). It uses Dockerode to communicate with the Docker daemon via the Unix socket. On startup, it cleans up any stale cloud-ide containers. The /start endpoint creates a new container with Traefik routing labels, so Traefik immediately begins routing userid.localhost traffic to that container's port 9000 without any manual configuration.

Aspect	Detail / Explanation
File	packages/traefik/src/index.ts
Runtime	Node.js on host machine — accesses /var/run/docker.sock
Library	Dockerode — Node.js Docker API client
Label routing	Traefik labels set at container creation → auto-discovered by Traefik Docker provider
Cleanup	Startup cleanup removes leftover containers from previous crashes
/start	Check image → pull if missing → createContainer (with labels) → container.start()
/running	listContainers({all:false}) → filter by name → boolean result
/stop	getContainer(userId) → stop() → remove()
Port	8080 — called by Next.js API routes /api/docker/start|stop|running

📄 **packages/traefik/src/index.ts** [TypeScript (Node.js)]

```
1  // packages/traefik/src/index.ts
2  // Management API (port 8080) — runs on the HOST, not inside a container.
3  // Uses Dockerode to manage user containers via /var/run/docker.sock
4  import Docker from "dockerode";
5  import express from "express";
6  import cors from "cors";
7
8  const docker = new Docker({ socketPath: "/var/run/docker.sock" });
9
10 // ——— Cleanup: remove all stale cloud-ide containers on startup ———
11 async function listStopAndRemoveCloudIdeContainers() {
12   const containers = await docker.listContainers({ all: true });
13   const cloudIdeContainers = containers.filter((c) =>
14     c.Image.startsWith("cloud-ide")
15   );
16   for (const containerInfo of cloudIdeContainers) {
17     const container = docker.getContainer(containerInfo.Id);
18     if (containerInfo.State === "running") {
```

```
      console.log(`Stopping: ${containerInfo.Names[0]}`);
      await container.stop();
    }
    console.log(`Removing: ${containerInfo.Names[0]}`);
    await container.remove();
  }
  console.log("Cleanup complete.");
}
listStopAndRemoveCloudIdeContainers().catch(console.error);

const managementAPI = express();
managementAPI.use(cors());
managementAPI.use(express.json());

// ——— POST /start — create & start a user's dedicated container ———
// Sets Traefik labels so the proxy auto-discovers the route:
//   Host(`{userId}.localhost`) → container port 9000
managementAPI.post("/start", async (req, res): Promise<any> => {
  const image = "cloud-ide";
  const tag   = "latest";

  // Pull image if not available locally
  let imageAlreadyExist = false;
  const images = await docker.listImages();
  for (const systemImage of images) {
    for (const systemTag of systemImage.RepoTags || []) {
      if (systemTag === `${image}:${tag}`) { imageAlreadyExist = true; break; }
    }
    if (imageAlreadyExist) break;
  }
  if (!imageAlreadyExist) await docker.pull(`${image}:${tag}`);

  // Create container with Traefik routing labels
  const container = await docker.createContainer({
    Image: `${image}:${tag}`,
    Tty:   false,
    name:  `${req.body.userId}`,
    Labels: {
      "traefik.enable": "true",
      [`traefik.http.routers.${req.body.userId}.rule`]:
        `Host(\`${req.body.userId}.localhost\`)`,
      [`traefik.http.routers.${req.body.userId}.entrypoints`]: "web",
      [`traefik.http.services.${req.body.userId}.loadbalancer.server.port`]:
        "9000",
    },
```

```
64   });
65
66   await container.start();
67   return res.json({
68     status:    "success",
69     container: `${(await container.inspect()).Name}.localhost`,
70   });
71 });
72
73 // ——— POST /running — check if a user's container is alive ——————————
74 managementAPI.post("/running", async (req, res): Promise<any> => {
75   const { userId } = req.body;
76   const containers = await docker.listContainers({ all: false });
77   const isRunning  = containers.some((c) => c.Names.includes(`/${userId}`));
78   return res.json({ running: isRunning });
79 });
80
81 // ——— POST /stop — stop and remove a user's container ————————————————
82 managementAPI.post("/stop", async (req, res): Promise<any> => {
83   const { userId }  = req.body;
84   const container   = await docker.getContainer(`${userId}`);
85   await container.stop();
86   await container.remove();
87   return res.json({ status: "success" });
88 });
89
90 managementAPI.listen(8080, () => {
91   console.log("Management API listening on port 8080");
92 });
```

SECTION H — CLOUD IDE: INFRASTRUCTURE

H.1 packages/server/Dockerfile — Container Image Definition

The Dockerfile builds the image that runs inside each user's container. It starts from node:22-alpine for a minimal footprint. System tools (curl, bash, python3, make, g++, build-base) are required by node-pty which compiles native C++ bindings. The python symlink ensures node-gyp finds python correctly. The server source code is copied into /home/server/ and npm install compiles the native modules.

Aspect	Detail / Explanation
File	packages/server/Dockerfile
Base	node:22-alpine — smallest viable Node.js image with apk package manager

Build tools	apk add build-base g++ make python3 — required for node-pty native compilation
Symlink	ln -sf python3 python — node-gyp calls 'python' but alpine only has python3
Port	EXPOSE 9000 — documented port for Docker and Traefik service label
CMD	npm run dev — starts server.ts via tsx or ts-node

packages/server/Dockerfile [Dockerfile]

```
1  # packages/server/Dockerfile
2  # Image for each user's Cloud IDE container.
3  # Based on Node 22 Alpine for minimal size.
4  # node-pty requires native build tools (python, make, g++).
5
6  FROM node:22-alpine
7
8  # Install system tools needed by node-pty and general dev use
9  RUN apk add --no-cache curl bash python3 make g++ build-base
10
11  # Symlink python3 → python (node-pty build scripts use 'python')
12  RUN ln -sf /usr/bin/python3 /usr/bin/python
13
14  # Set working directory
15  WORKDIR /home/server/
16
17  # Copy source (server.ts + package.json)
18  COPY . .
19
20  # Install Node.js dependencies (includes node-pty native compilation)
21  RUN npm install
22
23  # WebSocket + REST API port
24  EXPOSE 9000
25
26  # Start the TypeScript server via ts-node / tsx
27  CMD ["npm", "run", "dev"]
```

H.2 packages/traefik/docker-compose.yml — Service Orchestration

The docker-compose.yml starts two services: the Traefik reverse proxy and the Management API. Both mount /var/run/docker.sock — Traefik to watch container labels, and the Management API to control container lifecycle. Traefik uses the Docker provider, meaning it automatically discovers running containers by their labels with no additional configuration file. The Let's Encrypt resolver is pre-configured for easy HTTPS enablement in production.

Aspect	Detail / Explanation

File	packages/traefik/docker-compose.yml
Traefik image	traefik:v2.9 — stable release with Docker provider support
Docker socket	Mounted read-write on both services for label discovery and container control
network_mode	bridge — allows inter-service communication by container name
HTTPS	certificatesresolvers configured; uncomment HTTPS entrypoint for production
Management	reverse-proxy-app builds from local Dockerfile; exposes port 8080

packages/traefik/docker-compose.yml [YAML]

```yaml
# packages/traefik/docker-compose.yml
# Starts: 1) Traefik reverse proxy  2) Management API

services:

  # ── Traefik: dynamic reverse proxy ──────────────────────────────
  traefik:
    image: traefik:v2.9
    container_name: traefik
    command:
      # Enable Docker provider (watch container labels)
      - "--providers.docker=true"
      # HTTP entrypoint
      - "--entrypoints.web.address=:80"
      # HTTPS / Let's Encrypt (uncomment for production)
      # - "--entrypoints.web-secured.address=:443"
      - "--certificatesresolvers.myresolver.acme.tlschallenge=true"
      - "--certificatesresolvers.myresolver.acme.email=your-email@example.com"
      - "--certificatesresolvers.myresolver.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"       # HTTP traffic
      # - "443:443"   # HTTPS (enable for production)
      # - "8080:8080" # Traefik dashboard (disable in production)
    volumes:
      # Docker socket: Traefik reads labels from running containers
      - "/var/run/docker.sock:/var/run/docker.sock"
      # TLS cert storage (must be chmod 600 for Let's Encrypt)
      - "./letsencrypt:/letsencrypt"
    network_mode: bridge

  # ── Management API: handles container CRUD operations ──────────
  reverse-proxy-app:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
```

```
37      - "8080:8080"   # POST /start  POST /running  POST /stop
38    network_mode: bridge
39    volumes:
40      # Docker socket: Dockerode uses this to control containers
41      - /var/run/docker.sock:/var/run/docker.sock
```

SECTION I — CLOUD IDE: NEXT.JS API ROUTES

I.1 /api/docker/start|stop|running — Container Proxy Routes

The Cloud IDE Next.js frontend cannot call the Management API directly from the browser due to CORS and port restrictions. These three Next.js API routes act as server-side proxies: they receive the browser request, forward it to the Management API on localhost:8080, and return the response. This keeps the Management API port unexposed to the public internet.

Aspect	Detail / Explanation
Files	apps/client/src/app/api/docker/start|stop|running/route.ts
Pattern	Next.js Route Handler — POST method, server-side fetch to Management API
Security	Management API (port 8080) not exposed to browser; only reachable from Next.js server
/start	POST { userId } → creates and starts container → returns { status, container }
/stop	POST { userId } → stops and removes container → returns { status }
/running	POST { userId } → checks container liveness → returns { running: boolean }

📄 **apps/client/src/app/api/docker/[route]/route.ts** [TypeScript (Next.js)]

```
1  // apps/client/src/app/api/docker/start/route.ts
2  // Next.js API route: proxies start request to Management API
3  import { NextResponse } from "next/server";
4
5  export async function POST(req: Request) {
6    const body = await req.json();
7    try {

8      // Forward to Traefik management API running on host port 8080
9      const response = await fetch("http://localhost:8080/start", {
```

```
10       method:  "POST",
11       headers: { "Content-Type": "application/json" },
12       body:    JSON.stringify({ userId: body.userId }),
13     });
14     const data = await response.json();
15     return NextResponse.json(data);
16   } catch (error) {
17     return NextResponse.json(
18       { error: "Failed to start container" },
19       { status: 500 }
20     );
21   }
22 }
23
24 //
```

```
25 // apps/client/src/app/api/docker/stop/route.ts
26 import { NextResponse } from "next/server";
27
28 export async function POST(req: Request) {
29   const body = await req.json();
30   try {
31     const response = await fetch("http://localhost:8080/stop", {
32       method:  "POST",
33       headers: { "Content-Type": "application/json" },
34       body:    JSON.stringify({ userId: body.userId }),
35     });
36     const data = await response.json();
37     return NextResponse.json(data);
38   } catch (error) {
39     return NextResponse.json(
40       { error: "Failed to stop container" },
41       { status: 500 }
42     );
43   }
44 }
45
46 //
```

```
47 // apps/client/src/app/api/docker/running/route.ts
48 import { NextResponse } from "next/server";
49
50 export async function POST(req: Request) {
51   const body = await req.json();
52   try {
53     const response = await fetch("http://localhost:8080/running", {
54       method:  "POST",
55       headers: { "Content-Type": "application/json" },
56       body:    JSON.stringify({ userId: body.userId }),
```

```
57     });
58     const data = await response.json();
59     return NextResponse.json(data);   // { running: boolean }
60   } catch (error) {
61     return NextResponse.json({ running: false }, { status: 500 });
62   }
63 }
```

J.2 vstack-main/package.json — Dependencies Reference

The package.json lists all production and development dependencies for VStack. Key production dependencies include: Next.js 16.1.6, @google/generative-ai for Gemini AI, better-auth for authentication, drizzle-orm with postgres driver for database access, @paypal/react-paypal-js for payments, @codesandbox/sandpack-react for in-browser code execution, framer-motion for animations, and the full Radix UI / shadcn component suite.

Aspect	Detail / Explanation
Framework	next@16.1.6 with react@19 and react-dom@19
AI	@google/generative-ai@0.24.1
Auth	better-auth@1.5.4
Database	drizzle-orm@0.45.1 + postgres@3.4.8 + @neondatabase/serverless@1.0.2
Payments	@paypal/react-paypal-js@9.0.1
IDE Preview	@codesandbox/sandpack-react@2.20.0 + @codesandbox/sandpack-client@2.19.8
UI	Radix UI full suite + lucide-react@0.577.0 + framer-motion@12.35.1
Styling	tailwindcss@3.4.1 + tailwind-merge + tailwindcss-animate
Dev tools	TypeScript 5.9 + ESLint 10 + Prettier 3 + drizzle-kit
PackageMgr	pnpm@10.2.1 — workspace monorepo manager

END OF SOURCE CODE DOCUMENTATION

This document contains the complete, unabridged source code for all significant files in both the VStack AI website generator and the Cloud IDE browser-based development environment. The code is presented in dependency order with inline annotations to aid the examiner in understanding key design decisions.

For running the projects, refer to the README.md files in the enclosed CD. VStack requires a .env file with GEMINI_API_KEY, DATABASE_URL (Supabase), BETTER_AUTH_SECRET, and NEXT_PUBLIC_PAYPAL_KEY_ID. Cloud IDE requires Docker Engine and the cloud-ide:latest image to be built before running docker-compose up.

Aspect	Detail / Explanation
Total files documented	18 source files across both projects
VStack LoC (approx.)	~1,200 lines of TypeScript/TSX
Cloud IDE LoC (approx.)	~450 lines of TypeScript + infrastructure
Test coverage	Refer to Chapter 3.11 (Test Procedures) in main report
Full source	Pendrive attached to main project report — vstack-main.zip + Cloud-IDE-main.zip